

**МИНИСТЕРСТВО ТРАНСПОРТА РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ УНИВЕРСИТЕТ ТРАНСПОРТА»**

**ИНСТИТУТ ТРАНСПОРТНОЙ ТЕХНИКИ И СИСТЕМ УПРАВЛЕНИЯ
(ИТТСУ)**

Кафедра «Управление и защита информации»

Инструменты реверс-инжиниринга и транскомпиляции

Учебное пособие

Москва – 2024

**МИНИСТЕРСТВО ТРАНСПОРТА РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ УНИВЕРСИТЕТ ТРАНСПОРТА»**

**ИНСТИТУТ ТРАНСПОРТНОЙ ТЕХНИКИ И СИСТЕМ УПРАВЛЕНИЯ
(ИТТСУ)**

Кафедра «Управление и защита информации»

Инструменты реверс-инжиниринга и транскомпиляции

**Учебное пособие
по направлению подготовки
10.05.01 «Компьютерная безопасность»**

Москва – 2024

УДК 004.056.53

И 72

Инструменты реверс-инжиниринга и транскомпиляции: Учебное пособие / Ковров А.И., Ляпина Е.П., Савин Л.А., Сидоренко В.Г., Соколов И.Д. – М.: РУТ(МИИТ).2024. – 71 с.

В учебном пособии рассмотрены инструменты, реализующие функции дизассемблирования, декомпиляции и транскомпиляции и применяемые для анализа и синтеза программного кода, созданного на различных языках программирования высокого уровня.

Учебное пособие предназначено для обучающихся по специальности 10.05.01 «Компьютерная безопасность», а также других направлений, связанных с информационными технологиями.

Рецензенты:

Начальник центра искусственного интеллекта АО «ВНИИЖТ» М.А. Кулагин

Заведующий кафедрой «Вычислительные системы, сети и информационная безопасность» РУТ (МИИТ) Б.В. Желенков

© РУТ (МИИТ), 2024

Оглавление

Введение.....	4
1. Реверс-инжиниринг программного кода, созданного на языке программирования Python.....	6
2. Реверс-инжиниринг программного кода, созданного на языке программирования C#	18
2.1. Применение Инструмента ILSpy.....	18
2.2. Применение Инструмента .NET Reflector	24
3. Реверс-инжиниринг программного кода, созданного на языках программирования C и C++	27
3.1. Применение инструмента GHIDRA.....	27
3.2. Применение инструмента x64dbg	48
4. Конвертация программного кода с одного языка программирования на другой.....	54
4.1. Применение языковой модели с искусственным интеллектом в Telegram	54
4.2. Применение сервиса AI Code converter	62
Заключение	68
Список литературы	69

ВВЕДЕНИЕ

Инструментарий реверс-инжиниринга позволяет исследовать функционал программного обеспечения (ПО) и выявлять его недекларированные возможности. Делать это на основе анализа исходного или восстановленного программного кода на языке программирования (ЯП) высокого уровня удобнее, чем с использованием дизассемблированного кода. Получаемые при этом результаты скорее всего будут более полными. Возможности инструментов реверс-инжиниринга зачастую достаточно обширны. Среди них не только просмотр дизассемблированного кода, но и поиск строковых вхождений, построение графов программы, декомпиляция в высокоуровневые ЯП, плагины для скриптинга и многое другое. Выбор инструментария зависит от того, на каком ЯП было создано ПО. В учебном пособии рассмотрены инструменты, реализующие функции дизассемблирования и декомпиляции программного кода, созданного на различных ЯП высокого уровня, что позволяет провести сравнение особенностей результатов компиляции ПО и обратных к ней действий дизассемблирования и декомпиляции в зависимости от исходного ЯП.

В работах [1] проведена оценка применимости конкретного инструмента к исходному коду исполняемых файлов, созданному на конкретном ЯП с точки зрения читаемости дизассемблированного кода. Полученные результаты представлены в Таблице В.1.

Следует учесть, что программный код может быть дополнительно защищён с помощью обфускации – намеренного запутывания кода с помощью специального ПО. В таком случае специалисту сначала необходимо произвести обратный процесс, называемый деобфускация. Некоторые способы позволяют выполнить данный процесс «руками», однако более действенным решением будет использование дополнительного ПО [2].

В учебном пособии рассмотрен инструментариий конвертации исходного кода с одного ЯП на другой. Разбор программного кода основан на знании синтаксиса ЯП без учета бизнес-логики ПО. Такой подход можно

ограниченно применять для анализа программ. Основное же предназначение конвертации – продление жизнеспособности проектов, которые по тем или иным причинам устарели и нуждаются в обновлении.

Таблица В.1 – Сравнительная характеристика ЯП с точки зрения возможностей реверс-инжиниринга

Критерии сравнения	ЯП исходного кода программы		
	Python	C#	C++
Набор ПО для обратной разработки	PyInstaller Extractor, uncompyle6	.NET Reflector, ILSpy	IDA Pro, Radare2
Сложность анализа	Легкий	Средней сложности	Затруднённый
Возможность получить исходный код	Да	Частично	Нет
Уровень необходимых навыков для обратной разработки	Знание ЯП Python, базовые умения работы с командной строкой	Понимание логики промежуточного кода, умение работы с разнообразным ПО для обратной разработки. Навыки определения обфускации программного кода	Глубокие знания устройства ЯП ассемблера, логики ЯП C++, навыки работы с hex редакторами, разнообразным ПО для обратной разработки
Пригодность ЯП для использования в ПО систем управления	Нет	Пригоден в некоторых случаях	Рекомендуется к применению

Представленный в учебном пособии инструментарий применяется в ходе выполнения лабораторных работ по дисциплине «Технологии реверс-инжиниринга» и расширяет доступный обучающимся арсенал [3].

1. РЕВЕРС-ИНЖИНИРИНГ ПРОГРАММНОГО КОДА, СОЗДАННОГО НА ЯЗЫКЕ ПРОГРАММИРОВАНИЯ PYTHON

Рассмотрим возможный способ восстановления исходного кода ПО, разработанного на ЯП Python. Обобщенный алгоритм выглядит следующим образом:

1. Установка необходимых инструментов (PyInstaller Extractor, uncompyle6).
2. Перенос PyInstaller Extractor в папку с исполняемым файлом (*.exe), запуск со ссылкой на него.
3. Обработка файлов из созданной папки программой PyInstaller Extractor вручную и с помощью hex редактора.
4. Перевод байт-кода Python (*.pyc) в исходный код с использованием модуля uncompyle6.

Python – интерпретируемый ЯП (в отличие от С-подобных ЯП). Упаковку исходного кода программы в exe-файл выполняется с использованием дополнительных инструментов (например, PyInstaller, как показано на Рисунке 1.1).

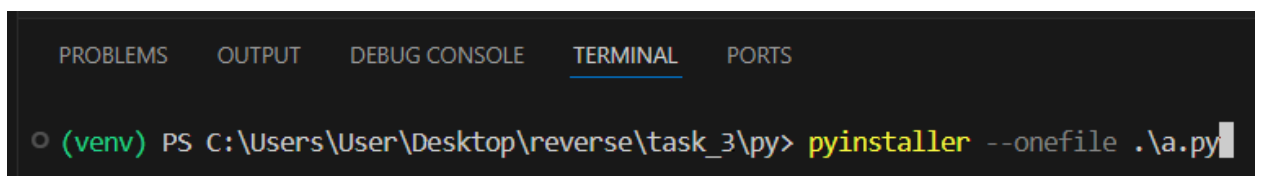


Рисунок 1.1 – Пример программы на Python

При запуске исполняемого файла создается новая папка, находящаяся по адресу user/AppData/Local/Temp и содержащая временные, необходимые для выполнения программы файлы. Данные файлы возможно найти после компиляции exe-файла.

ПРИМЕЧАНИЕ: при выполнении последующих действий лучше использовать версию Python до 3.10 (в данном примере используется версия

3.7.9), т.к. инструмент uncompryle6 (см. ниже) может работать некорректно на версиях выше.

Для того, чтобы не использовать папку Temp, создадим виртуальное окружение в папке с проектом с помощью команды `Python -m venv venv`. Активируется окружение командой `.\venv\Scripts\activate.bat`. После активации в терминале будет отображаться метка (venv), свидетельствующая о том, что окружение запущено. В этом случае после установки пакеты PyInstaller, uncompryle6 будут храниться в директории venv.

Для примера рассмотрим программу авторизации пользователя в системе (Рисунок 1.2).

```
def main(password, login):  
    user_login = input("Enter login: ")  
    user_password = input("Enter parol: ")  
    if (user_login == login) and (user_password == password):  
        print("Successful authorization")  
    else:  
        print("The entered data is incorrect")  
  
main("TKI", "UIZI")
```

Рисунок 1.2 – Пример программы на Python

Данная программа была упакована в exe-файл с помощью инструмента PyInstaller. Далее производится запуск exe-файла с помощью командной строки (Рисунок 1.3).

```
D:\Projects\pythonProject\dist>main.exe  
Enter login: |
```

Рисунок 1.3 – Запущенный exe-файла

После запуска в папке AppData\Local\Temp появляется новая папка, которая содержит в себе файлы, необходимые для выполнения exe-файла (Рисунок 1.4, 1.5).

Этот компьютер > Windows (C:) > Пользователи > Илья > AppData > Local > Temp				
Имя	Дата изменения	Тип	Размер	
pycache	17.09.2023 18:27	Папка с файлами		
_MEI97322	18.09.2023 21:17	Папка с файлами		
_MEI143282	23.09.2023 18:48	Папка с файлами		

Рисунок 1.4 – Расположение новой папки, созданной во время запуска exe-файла

Этот компьютер > Windows (C:) > Пользователи > Илья > AppData > Local > Temp > _MEI143282 >				
Имя	Дата изменения	Тип	Размер	
_bz2.pyd	23.09.2023 18:48	Python Extension ...	82 КБ	
_decimal.pyd	23.09.2023 18:48	Python Extension Module	245 КБ	
_hashlib.pyd	23.09.2023 18:48	Python Extension ...	61 КБ	
_lzma.pyd	23.09.2023 18:48	Python Extension ...	155 КБ	
_socket.pyd	23.09.2023 18:48	Python Extension ...	76 КБ	
_ssl.pyd	23.09.2023 18:48	Python Extension ...	156 КБ	
base_library.zip	23.09.2023 18:48	Сжатая ZIP-папка	1 042 КБ	
libcrypto-1_1.dll	23.09.2023 18:48	Расширение при...	3 359 КБ	
libssl-1_1.dll	23.09.2023 18:48	Расширение при...	683 КБ	
python310.dll	23.09.2023 18:48	Расширение при...	4 389 КБ	
select.pyd	23.09.2023 18:48	Python Extension ...	29 КБ	
unicodedata.pyd	23.09.2023 18:48	Python Extension ...	1 095 КБ	
VCRUNTIME140.dll	23.09.2023 18:48	Расширение при...	97 КБ	

Рисунок 1.5 – Дополнительные файлы, созданные в новой папке при запуске exe-файла

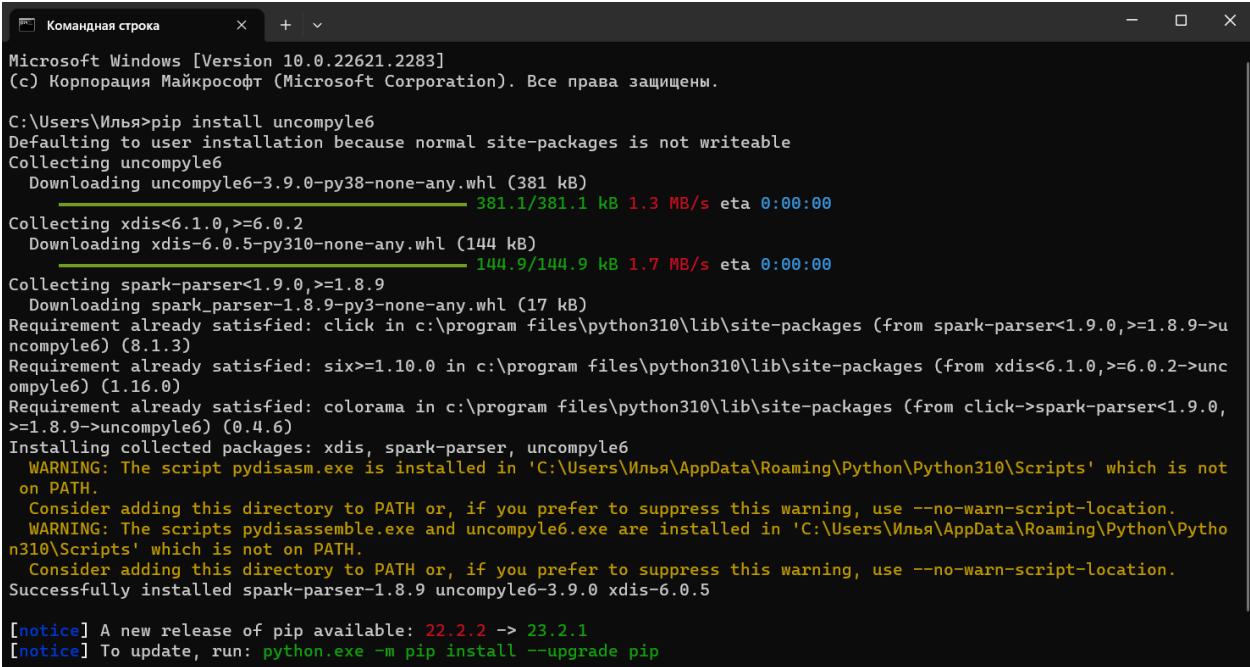
Во время запуска exe-файла распаковываются все библиотеки и коды в данную папку, а значит для их получения, а также ключей, лицензий, паролей, содержащихся в программе, достаточно обработать те файлы, которые были созданы при этом. Это говорит о том, что программы,

созданные на ЯП Python и упакованные с помощью PyInstaller Extractor, не защищены от получения исходного кода. Любой пользователь с достаточными знаниями сможет извлечь исполняемые файлы из папки, созданной во время запуска. Дополнительные меры защиты не имеют смысла, т.к. злоумышленник, у которого будет исполняемый файл, сможет получить доступ к исходному коду программы.

Следующий шаг – установка специального вспомогательного инструмента PyInstaller Extractor, который при запуске exe-файла извлечет все созданные файлы из папки Temp и сохранит их, чтобы в дальнейшем возможно было работать с ними уже без необходимости запускать exe-файл. Скачать инструмент можно по следующей ссылке:

<https://sourceforge.net/projects/pyinstallerextractor/>.

Еще понадобится дополнительный модуль uncompyle6, который можно установить с помощью команды `pip install` (Рисунок 1.6). uncompyle6 переводит байт-код Python обратно в эквивалентный исходный код Python.



```
Командная строка
Microsoft Windows [Version 10.0.22621.2283]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\Илья>pip install uncompyle6
Defaulting to user installation because normal site-packages is not writeable
Collecting uncompyle6
  Downloading uncompyle6-3.9.0-py38-none-any.whl (381 kB)
    381.1/381.1 kB 1.3 MB/s eta 0:00:00
Collecting xdis<6.1.0,>=6.0.2
  Downloading xdis-6.0.5-py310-none-any.whl (144 kB)
    144.9/144.9 kB 1.7 MB/s eta 0:00:00
Collecting spark-parser<1.9.0,>=1.8.9
  Downloading spark_parser-1.8.9-py3-none-any.whl (17 kB)
Requirement already satisfied: click in c:\program files\python310\lib\site-packages (from spark-parser<1.9.0,>=1.8.9->uncompyle6) (8.1.3)
Requirement already satisfied: six>=1.10.0 in c:\program files\python310\lib\site-packages (from xdis<6.1.0,>=6.0.2->uncompyle6) (1.16.0)
Requirement already satisfied: colorama in c:\program files\python310\lib\site-packages (from click->spark-parser<1.9.0,>=1.8.9->uncompyle6) (0.4.6)
Installing collected packages: xdis, spark-parser, uncompyle6
  WARNING: The script pydisasm.exe is installed in 'C:\Users\Илья\AppData\Roaming\Python\Python310\Scripts' which is not on PATH.
    Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
  WARNING: The scripts pydisassemble.exe and uncompyle6.exe are installed in 'C:\Users\Илья\AppData\Roaming\Python\Python310\Scripts' which is not on PATH.
    Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed spark-parser-1.8.9 uncompyle6-3.9.0 xdis-6.0.5

[notice] A new release of pip available: 22.2.2 -> 23.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Рисунок 1.6 – Установка uncompyle6

Далее необходимо переместить PyInstaller Extractor в папку с exe-файлом (Рисунок 1.7).

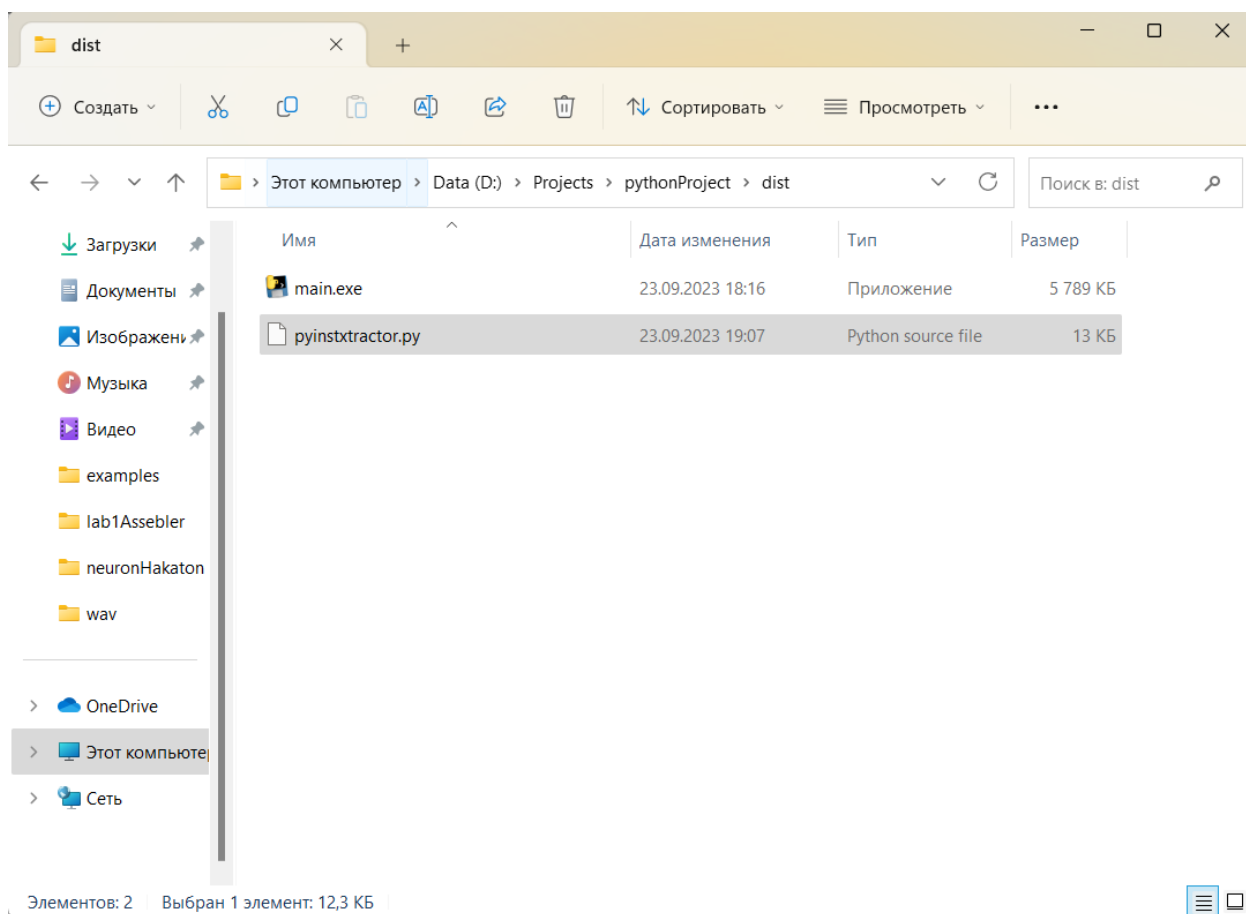


Рисунок 1.7 – Итоговое содержимое папки

Из этой папки в командной строке необходимо выполнить команду *python pyinstxtractor.py <имя файла>* (Рисунок 1.8).

После окончания работы PyInstaller Extractor в папке с ним и exe-файлом создается новая папка, в которой сохранились исполняемые файлы, созданные при запуске (Рисунки 1.9, 1.10).

```
Командная строка
Microsoft Windows [Version 10.0.22621.2283]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\Илья> D:

D:\>cd Projects\pythonProject\dist

D:\Projects\pythonProject\dist> python pyinstxtractor.py main.exe
D:\Projects\pythonProject\dist\pyinstxtractor.py:86: DeprecationWarning: the imp module is deprecated in favour of impor
tlib and slated for removal in Python 3.12; see the module's documentation for alternative uses
  import imp
[*] Processing main.exe
[*] Pyinstaller version: 2.1+
[*] Python version: 310
[*] Length of package: 5607337 bytes
[*] Found 22 files in CArchive
[*] Beginning extraction...please standby
[+] Possible entry point: pyiboot01_bootstrap
[+] Possible entry point: pyi_rth_inspect
[+] Possible entry point: main
[*] Found 97 files in PYZ archive
[*] Successfully extracted pyinstaller archive: main.exe

You can now use a python decompiler on the pyc files within the extracted directory

D:\Projects\pythonProject\dist>
```

Рисунок 1.8 – Запуск PyInstaller Extractor в командной строке

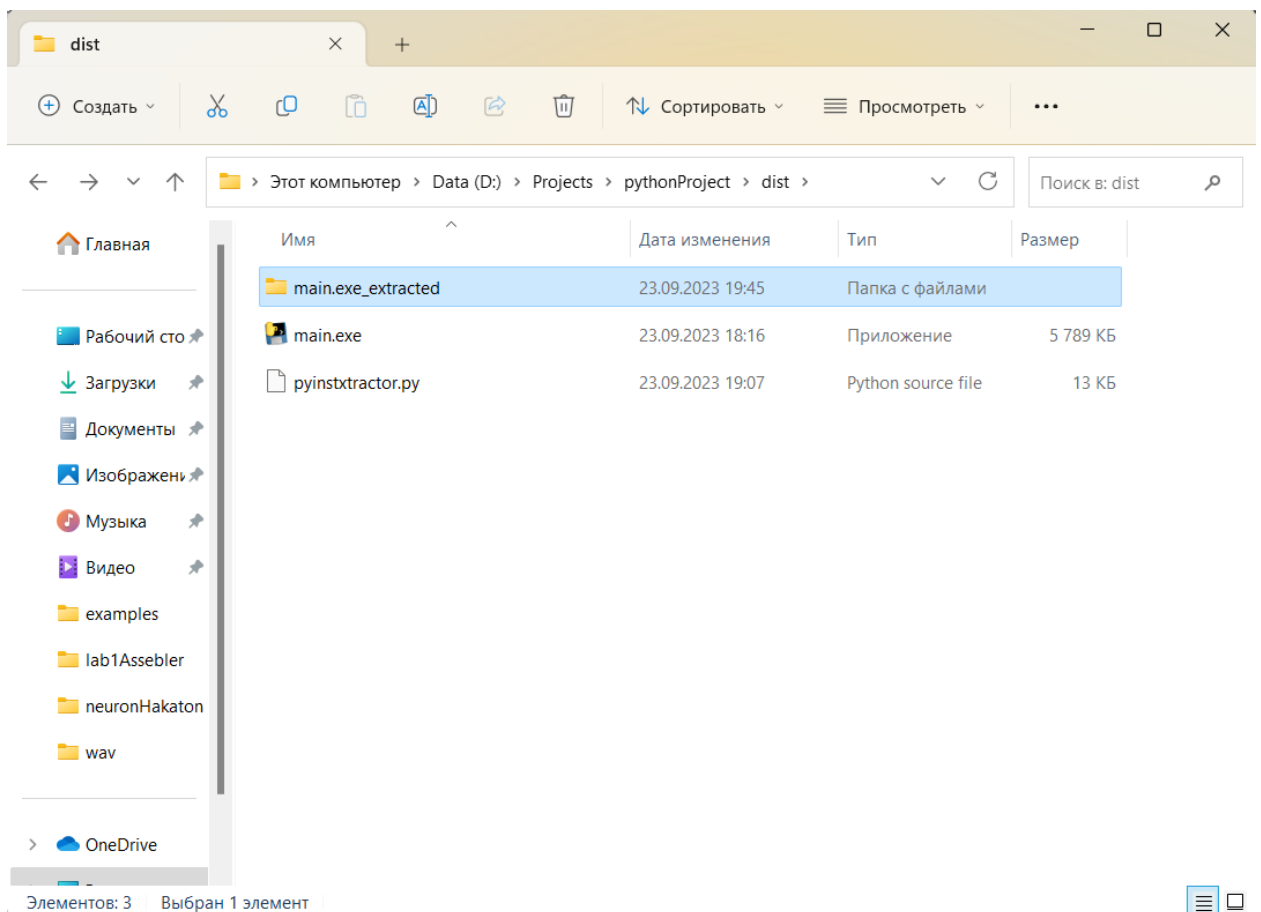


Рисунок 1.9 – Новая папка с файлами, созданными в ходе компиляции

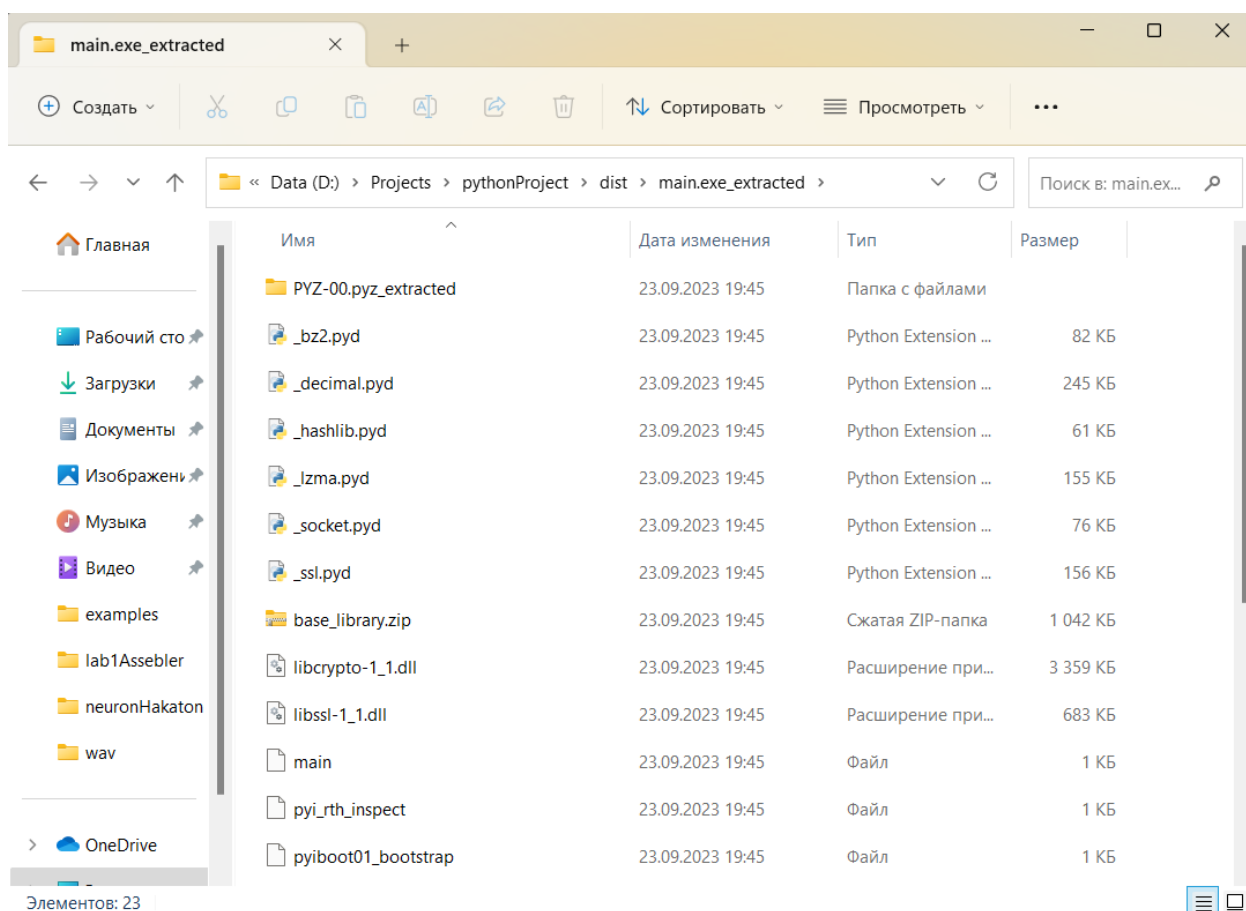


Рисунок 1.10 – Исполняемые файлы, созданные во время запуска exe-файла и сохраненные с помощью PyInstaller Extractor

Далее в новой папке необходимо найти файл с таким же названием, как и у исполняемого файла (Рисунок 1.11).

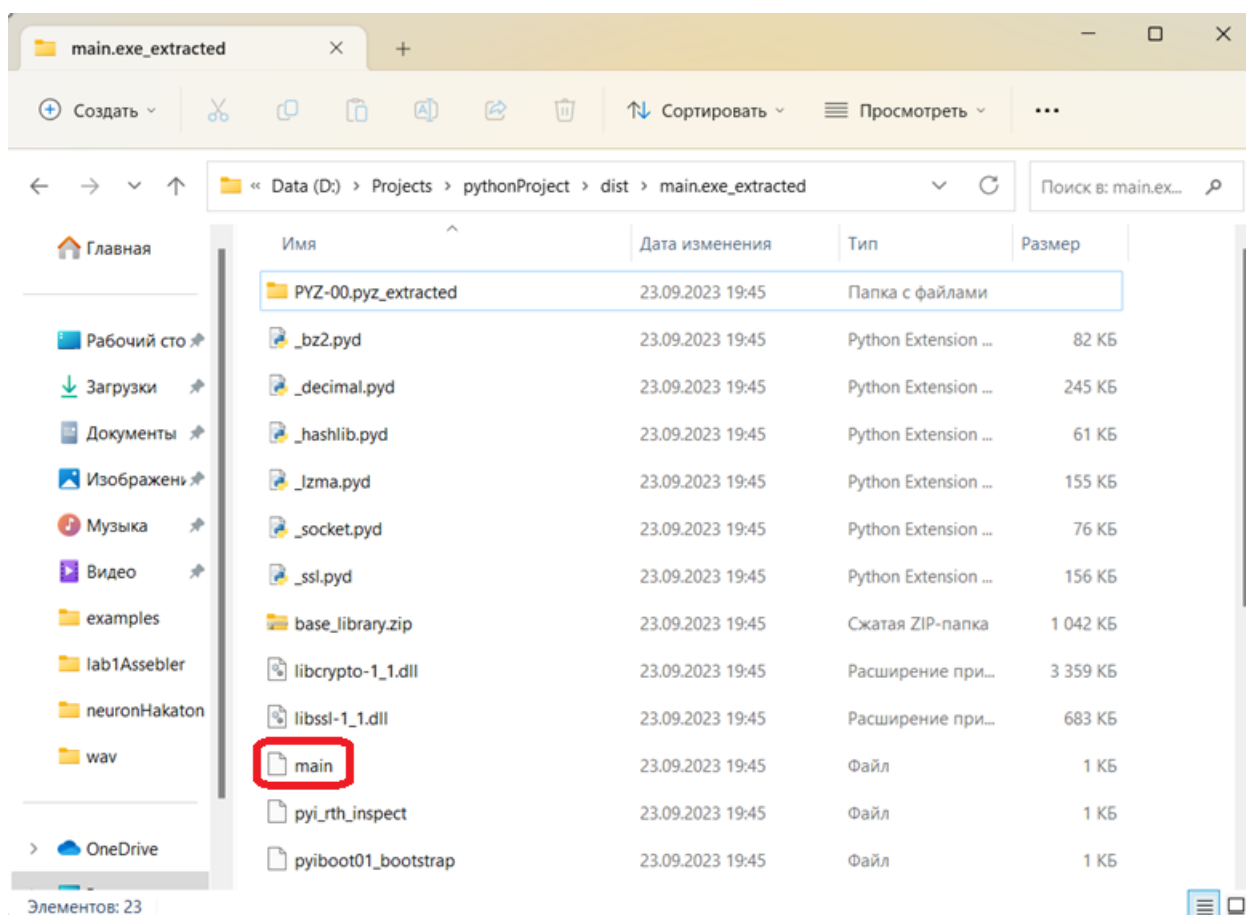


Рисунок 1.11 – Искомый файл

Далее необходимо задать формат .рус этому файлу (Рисунок 1.12).

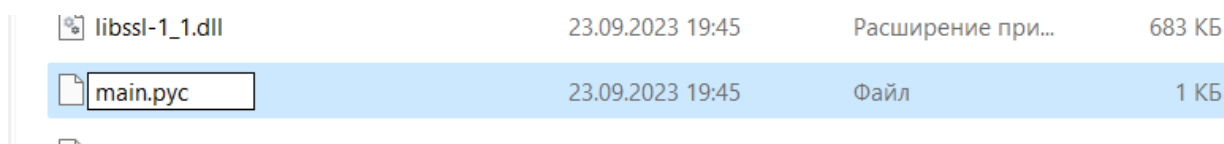


Рисунок 1.12 – Изменение формата файла

Далее в этой же папке необходимо найти архив base_library (Рисунок 1.13), в нем необходимо найти файл abs.рус (Рисунок 1.14).

Далее нужно открыть этот файл через hex редактор и скопировать первую строку (Рисунок 1.15).

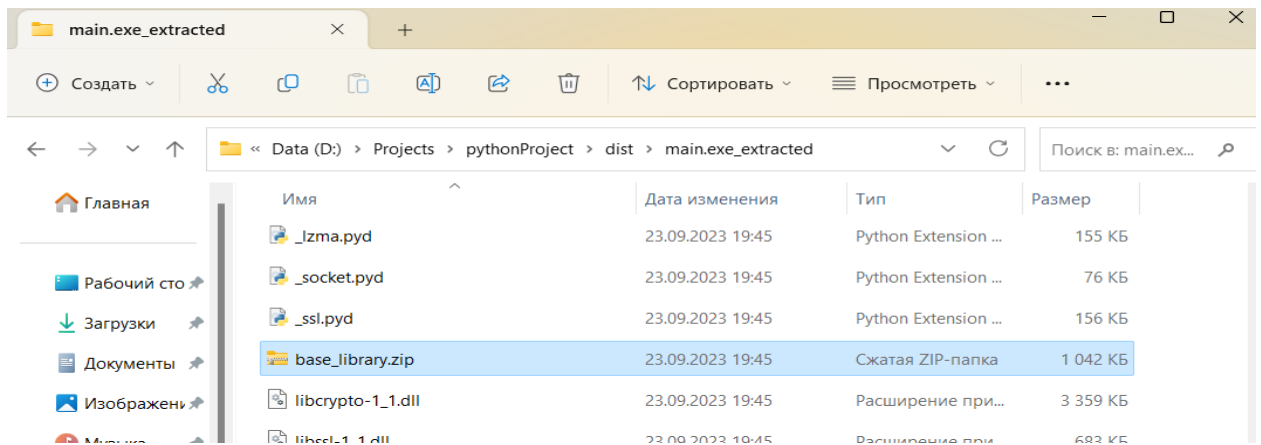


Рисунок 1.13 – Найденный архив

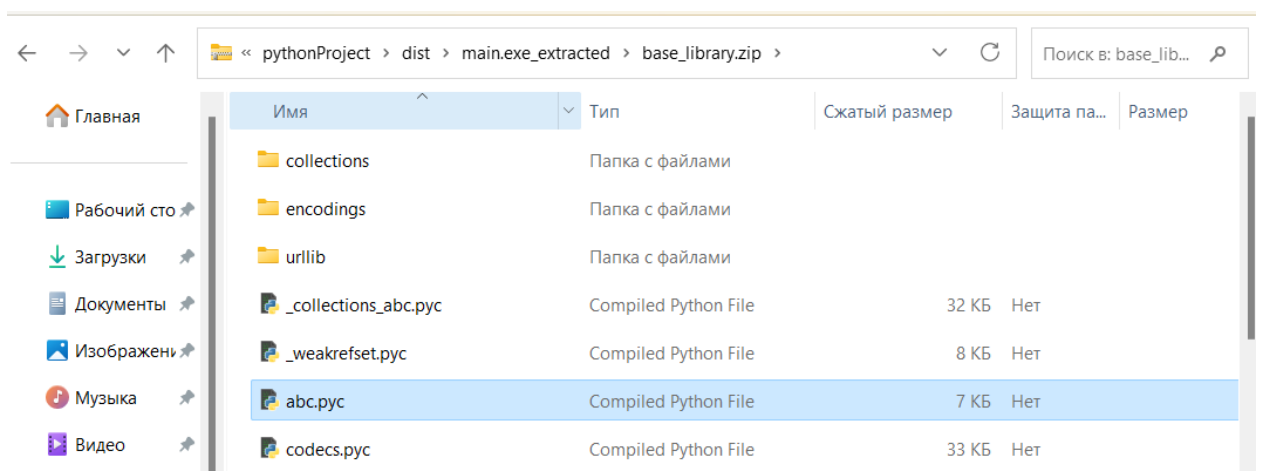


Рисунок 1.14 – Искомый файл в архиве

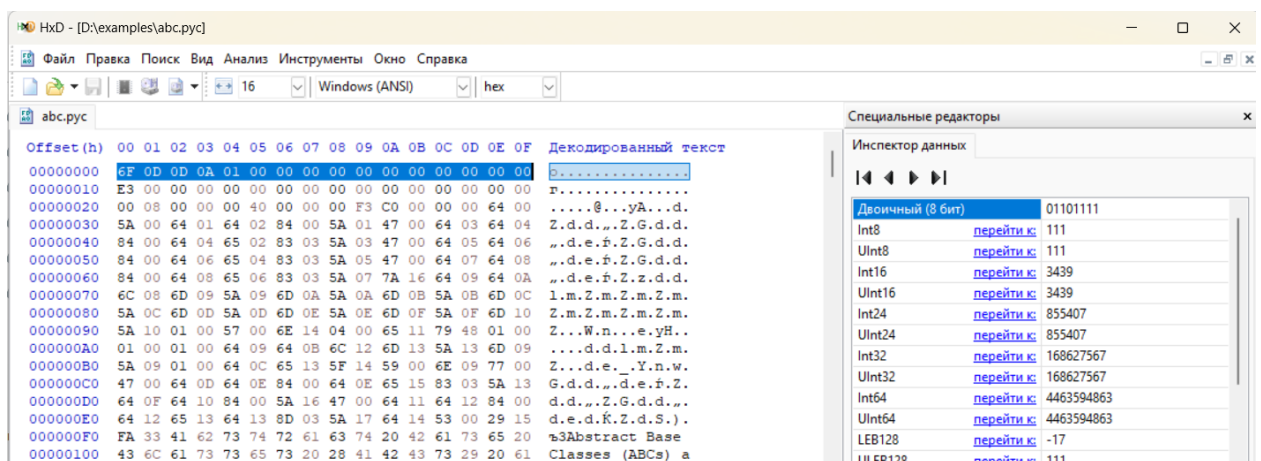


Рисунок 1.15 – Нех-редактор

Эту строку нужно вставить в файл .рус (Рисунки 1.10, 1.11) и сохранить (Рисунок 1.16).

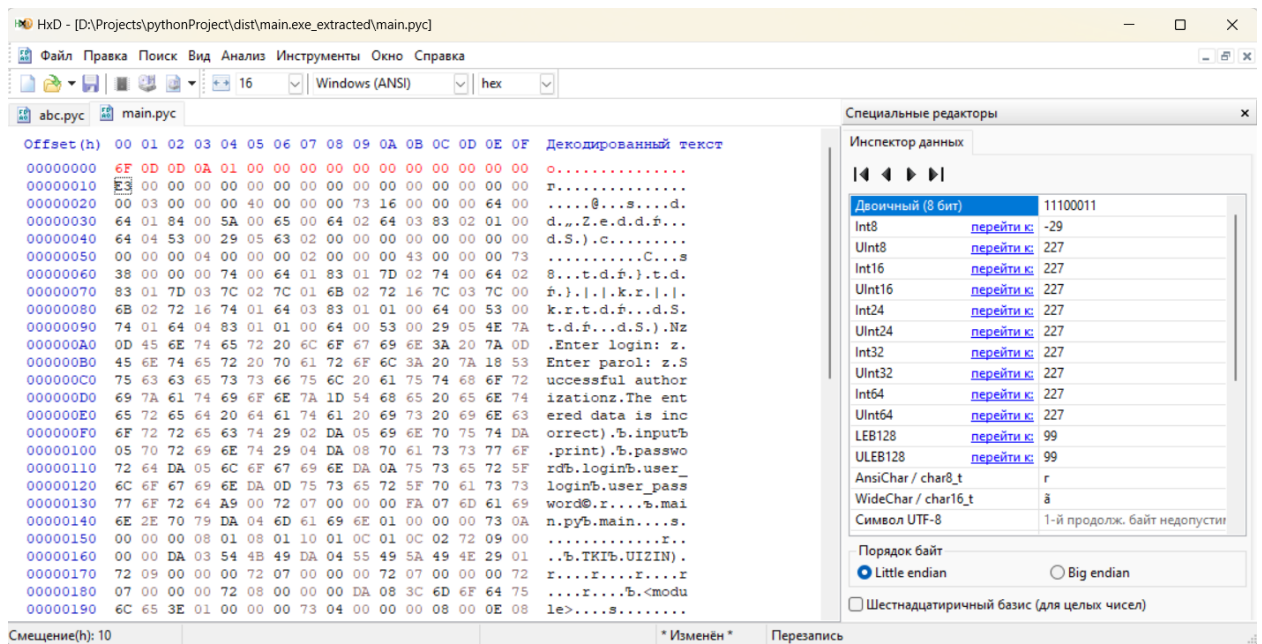


Рисунок 1.16 – Изменение файла main.pyc

Далее через консоль необходимо запустить uncompyl6 с указанием на файл main.pyc, с сохранение в новый файл, где будет находиться исходный код программы (Рисунок 1.17).

```
D:\Projects\pythonProject\dist\main.exe_extracted>uncompyl6 main.pyc > decoded_main.py
D:\Projects\pythonProject\dist\main.exe_extracted>
```

Рисунок 1.17 – Запуск uncompyl6

В папке появился новый файл с кодом на python (Рисунок 1.18).

Открыв этот файл в любой интегрированной среде разработки (integrated development environment, IDE, в данном примере использовалась Visual Studio 2019) можно увидеть исходный код на Python, написанный изначально, до упаковки exe-файла (Рисунок 1.19). Таким образом, удалось получить из exe-файла исходный код, т.е. выполнить процедуру реверс-инжиниринга.

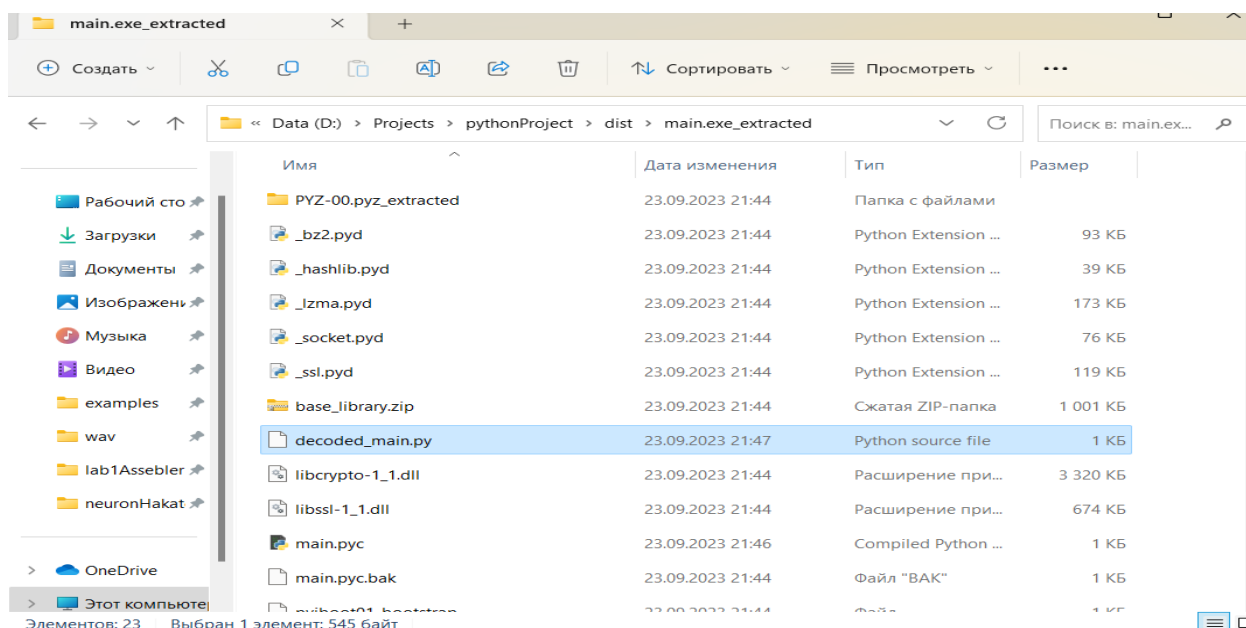


Рисунок 1.18 – Файл с исходным кодом

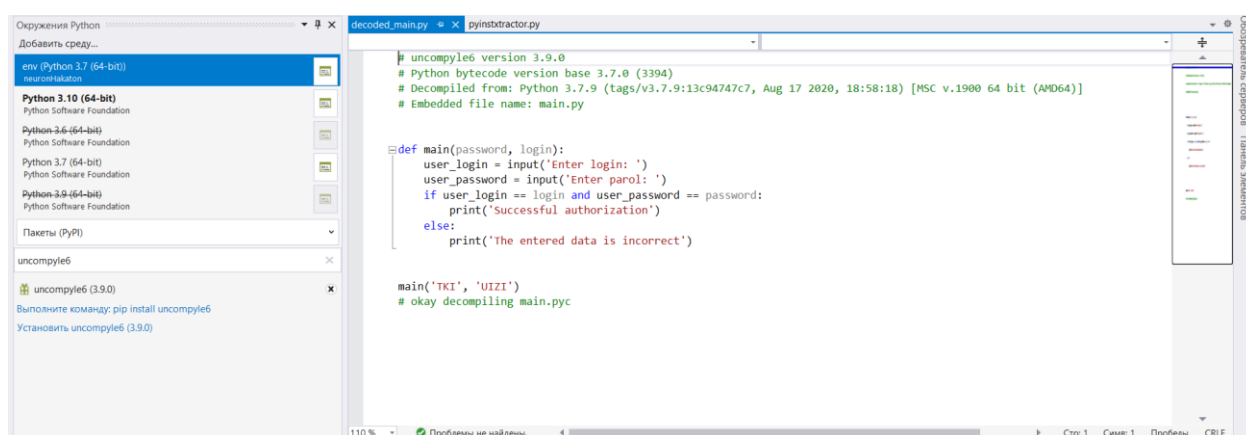


Рисунок 1.19 – Полученный исходный код

Таким образом, использовать Python для создания ПО небезопасно, т.к. исходный код можно получить без серьезного анализа (с помощью IDA или RADARE2), что делает бесполезным защиту внутри кода путем шифрования или хеширования. Для создания ПО более безопасно использовать ЯП C++ и ему подобные, т.к. они компилируемые и, следовательно, для и компиляции и запуска ПО системе не придется создавать новые вспомогательные файлы, которые можно обнаружить в папке Temp простым способом.

Инструментарий реверс инжиниринга постоянно развивается. Представленные в разделе инструменты применимы для ЯП Python до версии

3.10. Выполнять аналогичные действия для программного кода, созданного на более новых версиях, позволяет инструмент <https://github.com/zrax/pycdc>. Его можно собирать из репозитория или использовать готовую сборку.

Упакованный в exe-файл с использованием инструмента PyInstaller исходный код программы (Рисунке 1.1) был успешно извлечен в среде Linux (Рисунок 1.20).

```
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Установите последнюю версию PowerShell для новых функций и улучшения! https://aka.ms/PSWindows

PS C:\Users\Илья\OneDrive\Документы\test> python .\pyinstxtractor.py .\Programm.exe
C:\Users\Илья\OneDrive\Документы\test\pyinstxtractor.py:86: DeprecationWarning: the imp module is deprecated in favour of
importlib and slated for removal in Python 3.12; see the module's documentation for alternative uses
  import imp
[*] Processing .\Programm.exe
[*] Pyinstaller version: 2.1+
[*] Python version: 310
[*] Length of package: 5607434 bytes
[*] Found 22 files in CArchive
[*] Beginning extraction...please standby
[+] Possible entry point: pyiboot01_bootstrap
[+] Possible entry point: pyi_rth_inspect
[+] Possible entry point: Programm
[*] Found 97 files in PYZ archive
[*] Successfully extracted pyinstaller archive: .\Programm.exe

You can now use a python decompiler on the pyc files within the extracted directory
PS C:\Users\Илья\OneDrive\Документы\test> |
```

Рисунок 1.20 – Результат работы инструмента pycdc

2. РЕВЕРС-ИНЖИНИРИНГ ПРОГРАММНОГО КОДА, СОЗДАННОГО НА ЯЗЫКЕ ПРОГРАММИРОВАНИЯ C#

2.1. ПРИМЕНЕНИЕ ИНСТРУМЕНТА ILSPY

В данном блоке рассмотрим инструмент ILSpy, который позволяет декомпилировать приложения, написанные на ЯП C#.

Приступим к установке данного инструмента. Для этого необходимо скачать официальный репозиторий проекта с GitHub [4]. В данных методических указаниях рассмотрим установку и запуск при помощи Microsoft Visual Studio 2022 (MVS). После успешной загрузки найдём в папке файл «ILSpy.sln», который является файлом решения MVS. После успешной загрузки решения, MVS предложит установить необходимые зависимости, с чем мы соглашаемся (Рисунок 2.1).

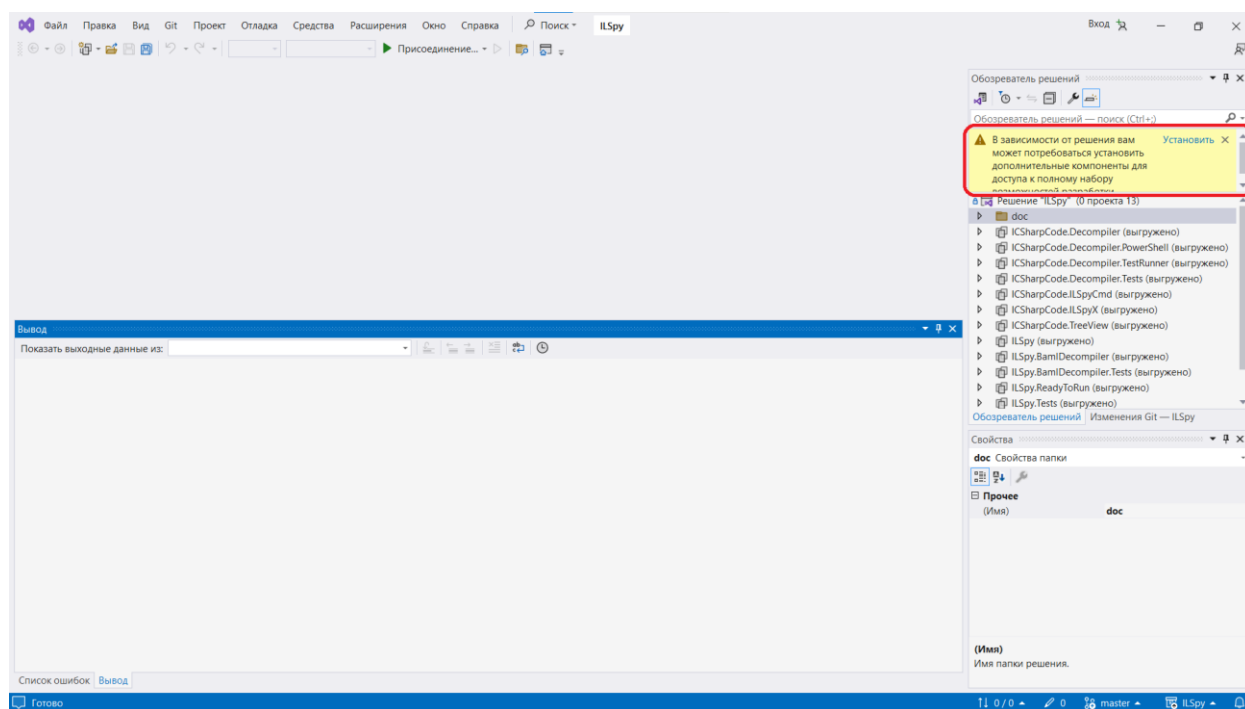


Рисунок 2.1 – Установка необходимых компонентов

Откроется программ установщика MVS, где необходимо выбрать кнопку установить (Рисунок 2.2).

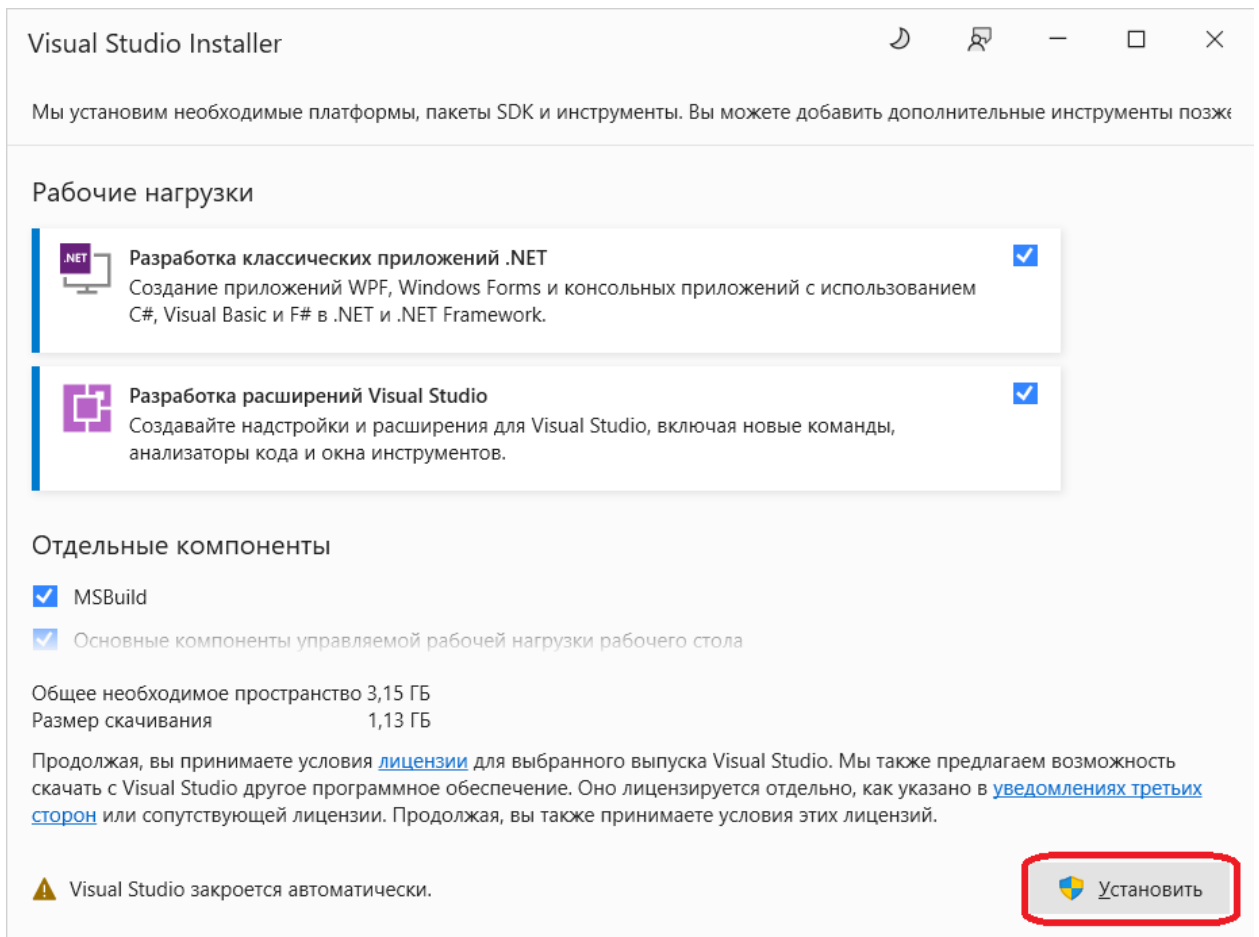


Рисунок 2.2 – Второй шаг установки компонентов

После успешной установки вернёмся к нашему проекту. Теперь, когда все зависимости разрешены, можно запускать приложение. Для этого необходимо нажать на зелёный треугольник в меню (Рисунок 2.3).

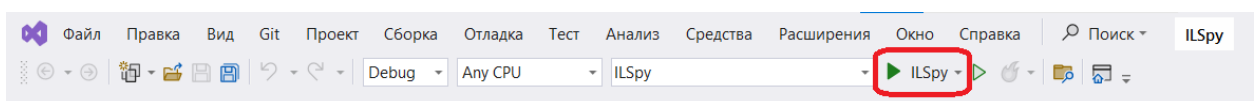


Рисунок 2.3 – Начало сборки проекта

После нажатия начнётся процесс сборки. Это может занять продолжительное время. По завершении всех операций откроется окно программы.

Последующие запуски будут осуществляться быстрее, так как программа уже сконфигурирует себя под конкретную систему.

Перейдём к основной части. Начнём процесс реверс-инжиниринга с запуска анализируемой программы с сайта, содержащего специально собранные программы для практики в искусстве реверс-инжиниринга, а конкретнее, программу CrackMe v.2 [5]. Пароль для архива с программой – «crackmes.one». В результате запуска появится окно, изображенное на Рисунке 2.4.

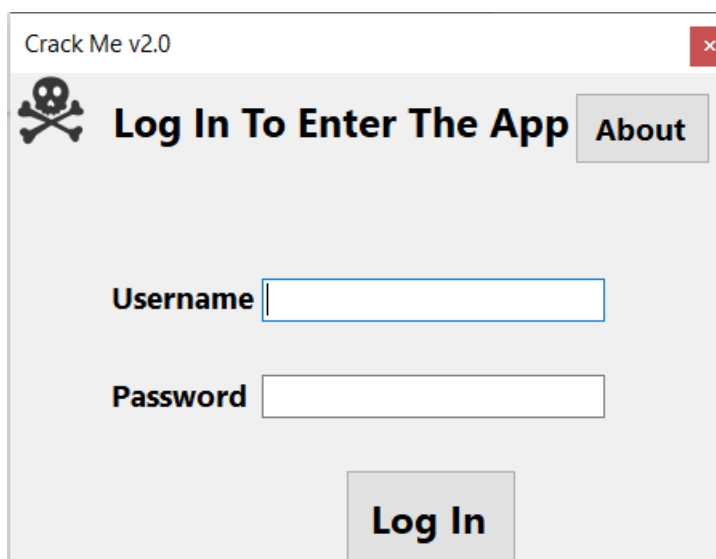


Рисунок 2.4 – Окно запущенной программы

Здесь мы видим два поля, в которые можно ввести данные, и кнопку «Log In». Проверим, что программа действительно не даёт произвести авторизацию со случайным логином-паролем (Рисунок 2.5).

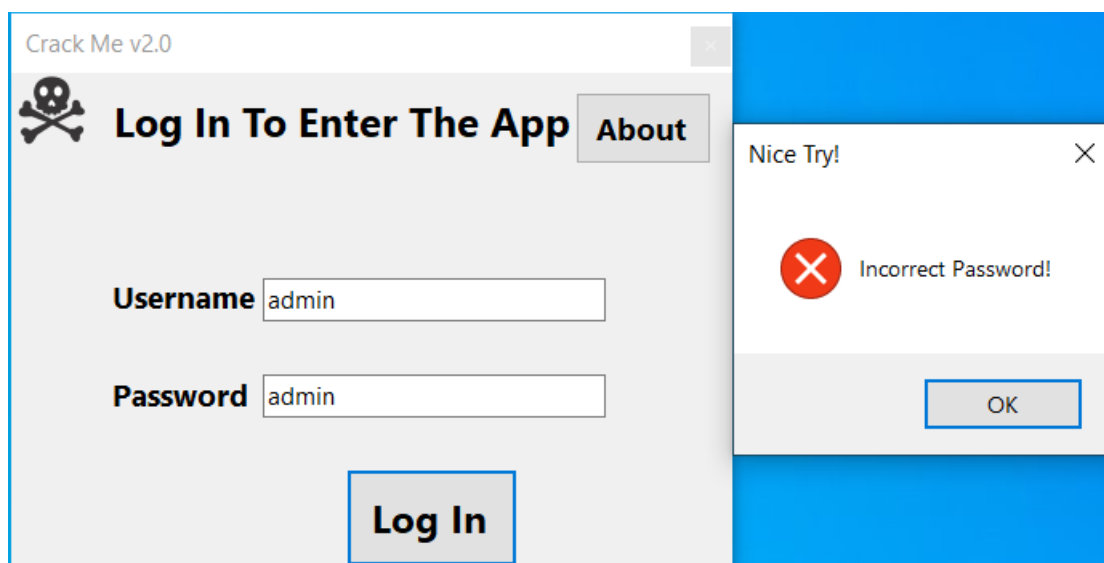


Рисунок 2.5 – Результат ввода неправильного пароля

Действительно, программа не позволяет просто так войти внутрь.

Программа ILSpy содержит большое число настроек, но наиболее интересны для нас выбор ЯП и его версии, на который программа будет декомпилировать код. Настройки по умолчанию – C#, и C# 11.0 соответственно, что вполне нас устраивает. Помимо этого, программа имеет несколько функциональных областей, с которыми мы будем работать, а именно: навигация по файлам и функциям (отмечено цифрой 1) и сам декомпилированный код (отмечено цифрой 2). Это отображено на Рисунке 2.6.

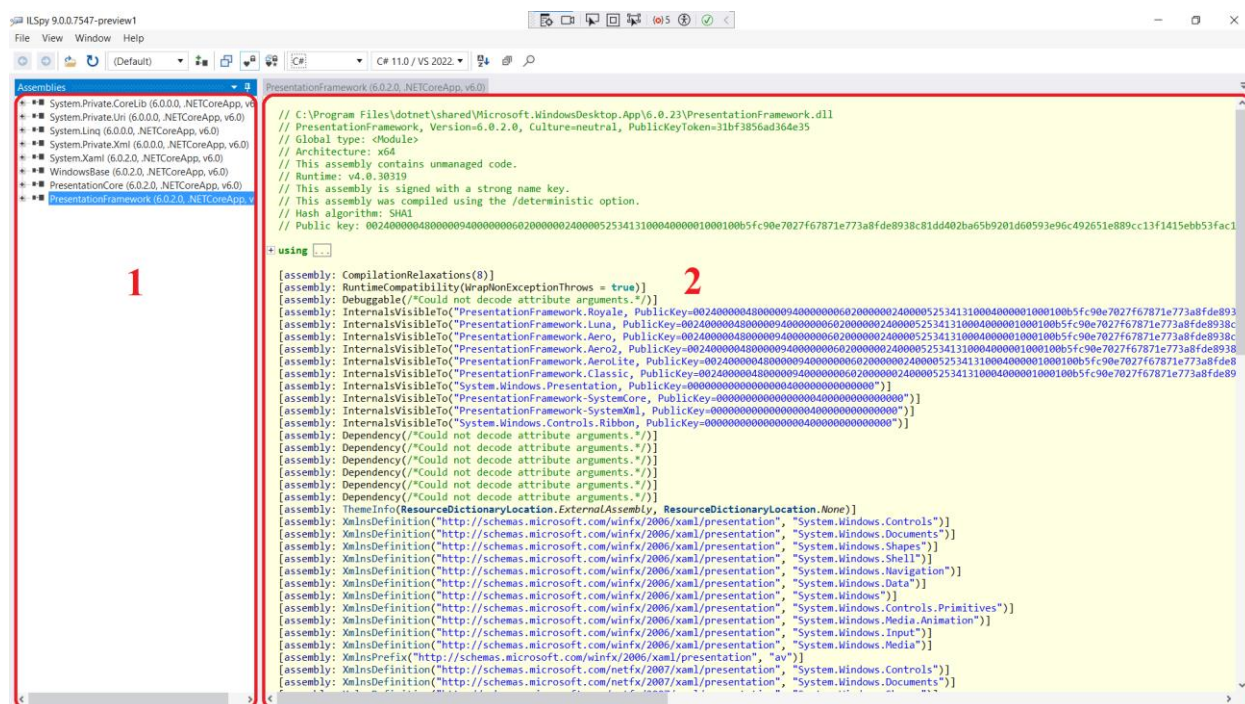


Рисунок 2.6 – Окно MVS

В меню навигации найдём класс Form1. В этом классе, как можно видеть на Рисунке 2.7, объявлены основные методы и поля нашего приложения.

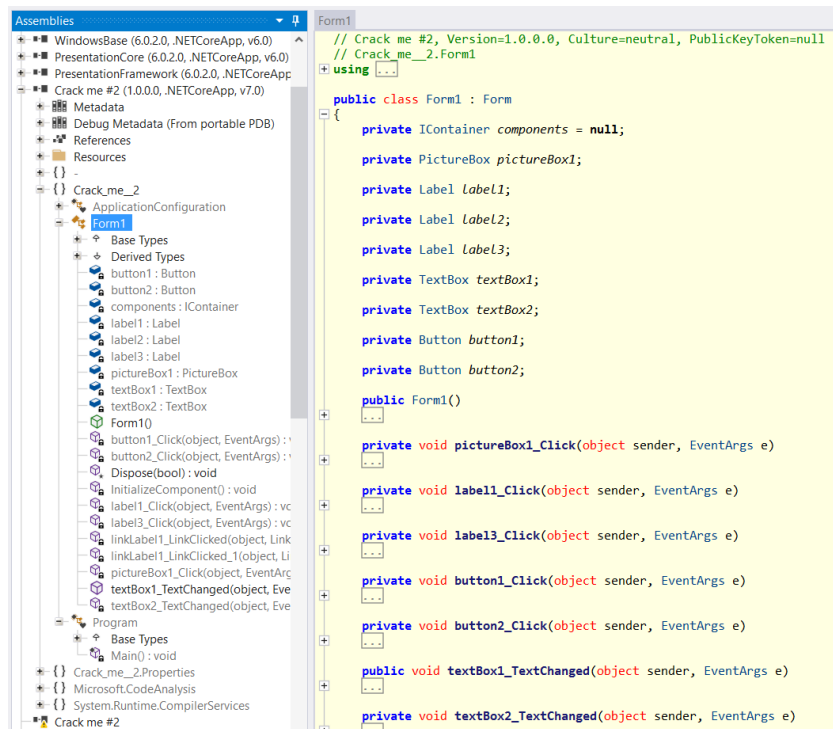


Рисунок 2.7 – Основные методы и поля MVS

Просмотрим методы этого класса. В методе button1_Click описан интересующий нас функционал, отражённый на Рисунке 2.8.

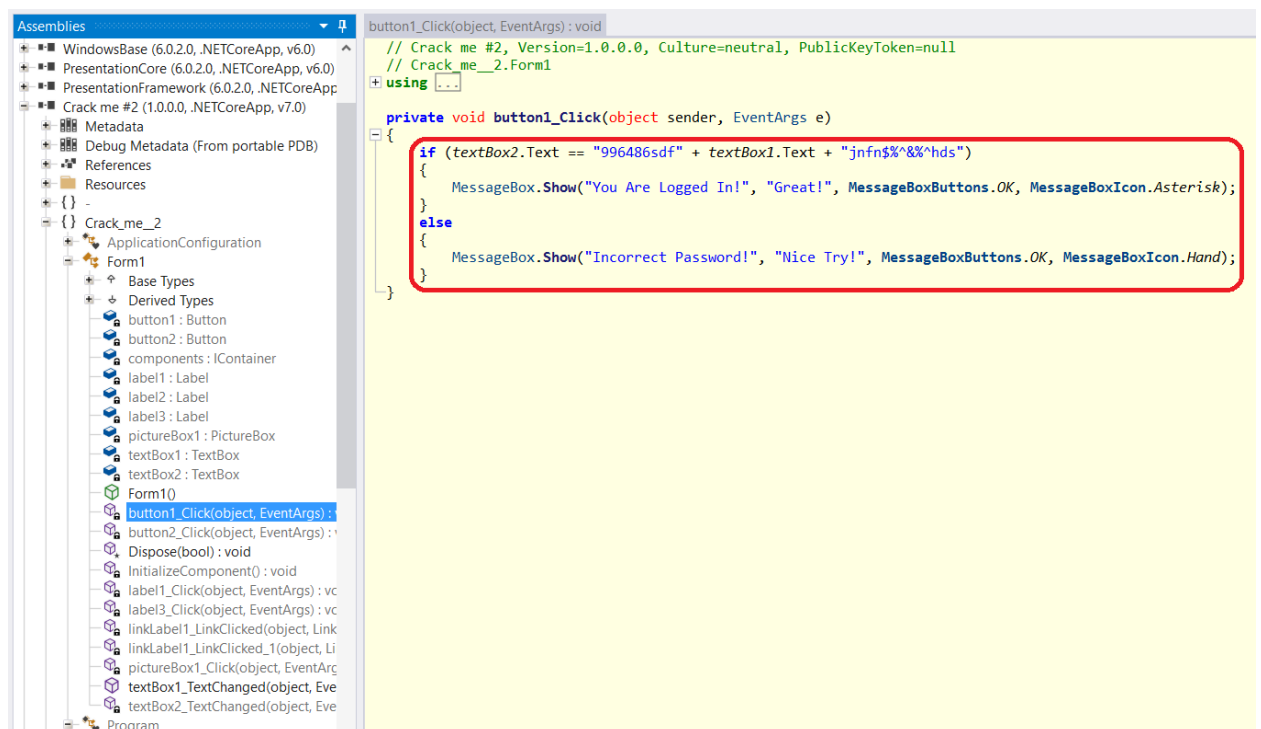


Рисунок 2.8 – Функционал авторизации

Данный код выполняет проверку на равенство содержимого текстового поля `textBox2` и строки `"996486sdf" + textBox1.Text + "jnfn$%^&%^hds"`, где `textBox1.Text` – содержимое поля `textBox1`. Если условие выполняется, выводится сообщение «You Are Logged In! Great!», а в противном случае: «Incorrect Password! Nice Try!». Таким образом, программа ожидает, что мы введём некоторый логин, а затем пароль в соответствии с заданной конструкцией, в которой используется логин. Проверим это. Введём в первое поле «admin», а во второе «996486sdfadminjnfn\$%^&%^hds». Результат отражён на Рисунке 2.9.

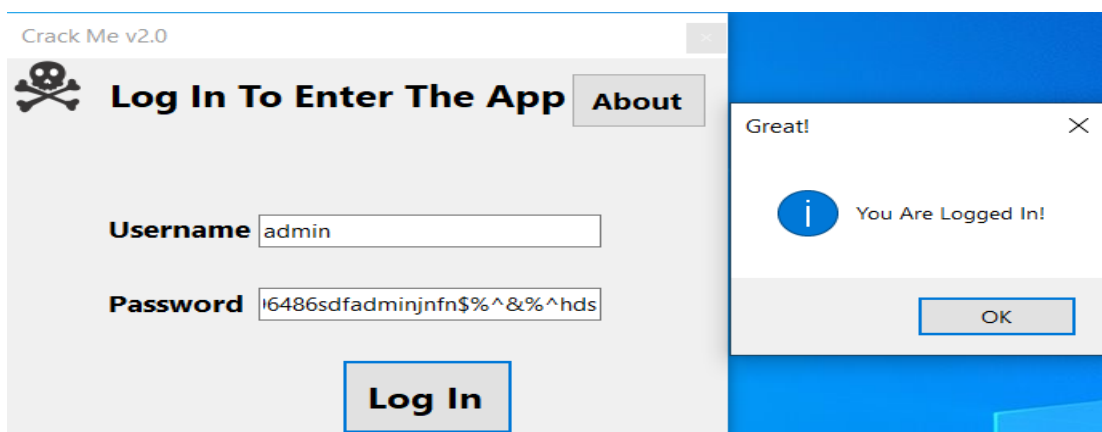


Рисунок 2.9 – Результат ввода правильного пароля

2.2. ПРИМЕНЕНИЕ ИНСТРУМЕНТА .NET REFLECTOR

В данном разделе рассмотрим процесс реверс-инжиниринга ПО, исходный код которого написан на ЯП C#, с использованием ПО .NET Reflector от разработчика Redgate [6]. В качестве примера разберем программу, которая была использована в предыдущем примере.

Запустим .NET Reflector. Главное меню программы представлено на Рисунке 2.10.

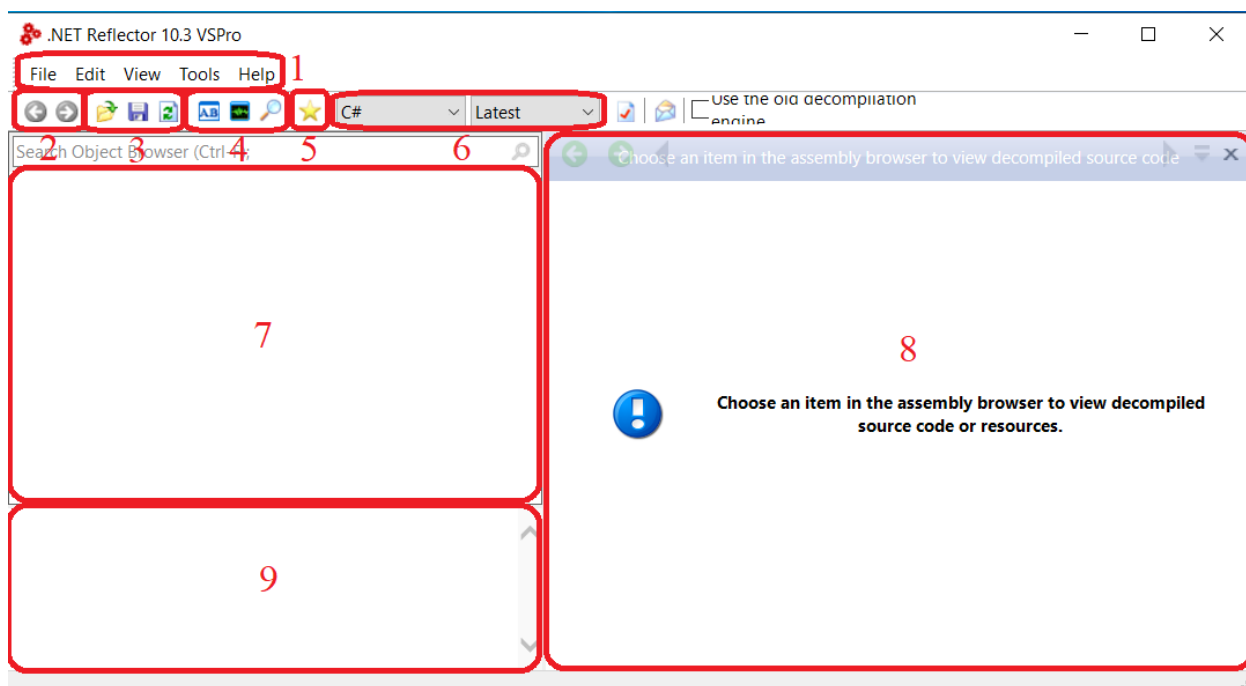


Рисунок 2.10 – Главное меню программы NET Reflector

В главном меню программы расположены несколько позиций:

Блок 1 – это основное контекстное меню, предоставляющее доступ ко всему функционалу программы.

Блок 2 отвечает за навигацию по файлу и по коду.

Блок 3 отвечает за загрузку, сохранение и обновление полей программы.

Блок 4 содержит в себе инструменты декомпиляции, анализа и поиска.

Блок 5 позволяет перейти в закладки в коде, которые пользователь сам определит.

Блок 6 позволяет выбрать ЯП декомпиляции программы и версию.

Блок 7 отвечает за список загруженных файлов и навигацию по ним. Здесь можно перейти в методы и функции программы, что будет рассмотрено далее.

Блок 8 позволяет нам просмотреть декомпилированный код, где мы, собственно, и будем работать.

Блок 9 отвечает за вывод информации о файле или библиотеке.

Для того, чтобы начать работу, откроем библиотеку нашего приложения в программе. Для этого выберем .dll файл, который необходим для работы данной программы, он находится в той же папке, что и программа. Пройдёмся по файлу в поиске «интересных» методов. Как видим, в программе есть класс Form1, который содержит основные поля и методы программы (Рисунок 2.11). Полученные результаты аналогичны результатам представленным на Рисунке 2.7. Как показала практика, ключевой функционал находится в подобных классах. Пройдём глубже.

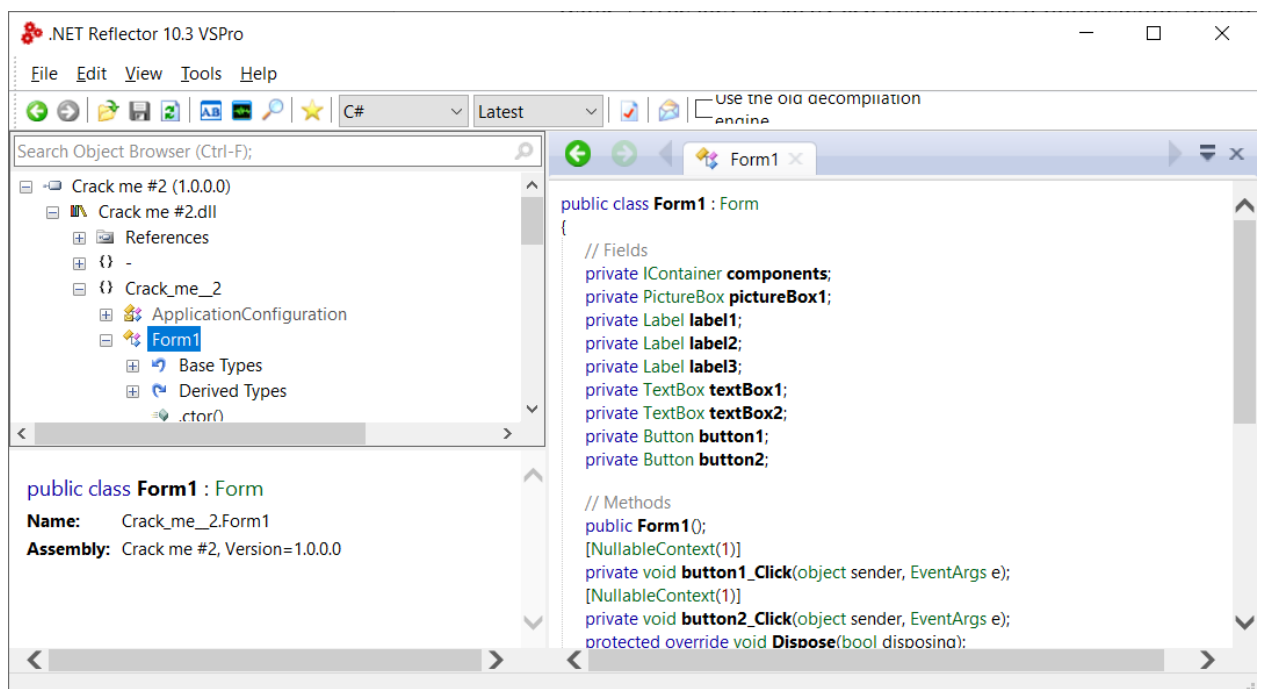


Рисунок 2.11 – Класс Form1

Зайдём в метод обработки нажатия `button1_Click`. Как можно видеть, внутри находится декомпилированный код метода, и он содержит несколько полезных нам строк (Рисунок 2.12). аналогичных представленным на Рисунке 2.8.

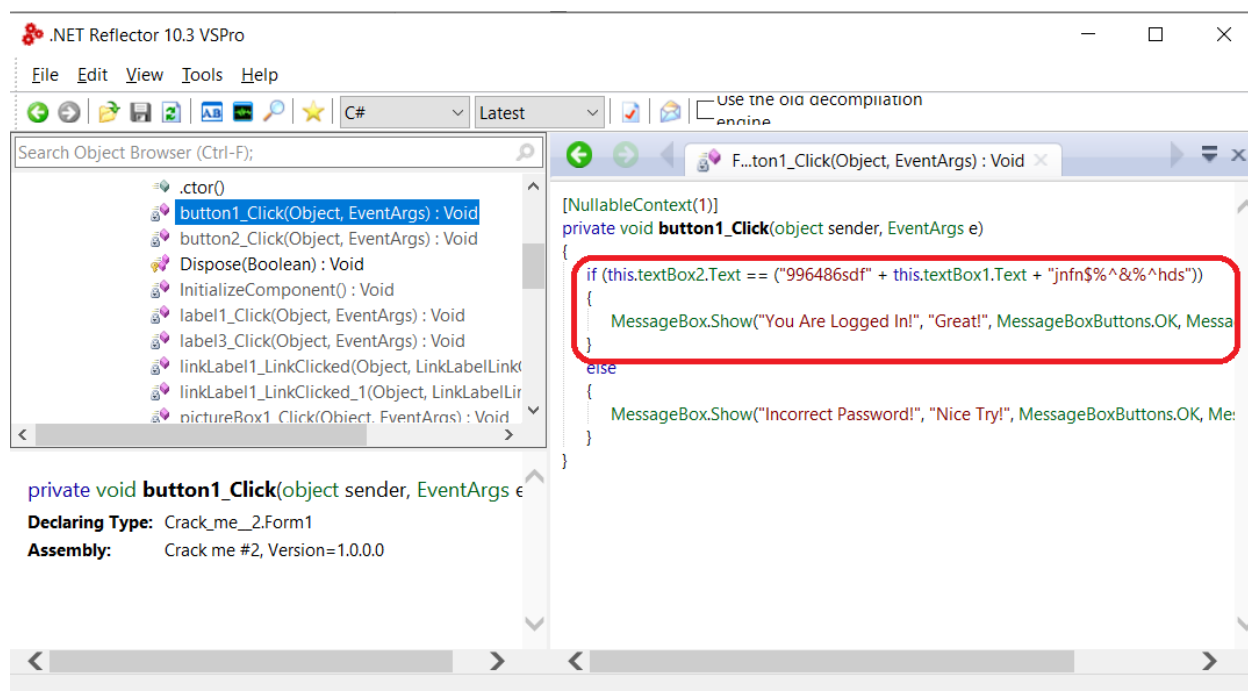


Рисунок 2.12 – Метод `button1_Click`

Результат успешного входа в программу совпадает с представленным на Рисунке 2.9.

Таким образом, написание программ на ЯП С# является небезопасной практикой, т.к. любой человек, имеющий необходимые минимальные знания в этой области и подходящее ПО, способен получить и проанализировать исходный код любой программы.

3. РЕВЕРС-ИНЖИНИРИНГ ПРОГРАММНОГО КОДА, СОЗДАННОГО НА ЯЗЫКАХ ПРОГРАММИРОВАНИЯ C И C++

3.1. ПРИМЕНЕНИЕ ИНСТРУМЕНТА GHIDRA

В данном разделе рассмотрен инструмент отладки программ под названием GHIDRA. В основные возможности этой программы входит декомпиляция проектов, анализ кода, построение разнообразных графов для анализа программ. В целом, инструмент является открытым и легко расширяемым, что даёт возможность любому пользователю написать плагин, реализующий функционал под его нужды, и интегрировать его в среду GHIDRA. Данное ПО позволяет выполнять удалённое подключение к серверу, что даёт возможность сразу нескольким людям работать над одним проектом. Перейдём к установке данной программы.

Первым шагом необходимо скачать архив, содержащий программу, с сайта GitHub [7]. Нужен файл, указанный на Рисунке 3.1.

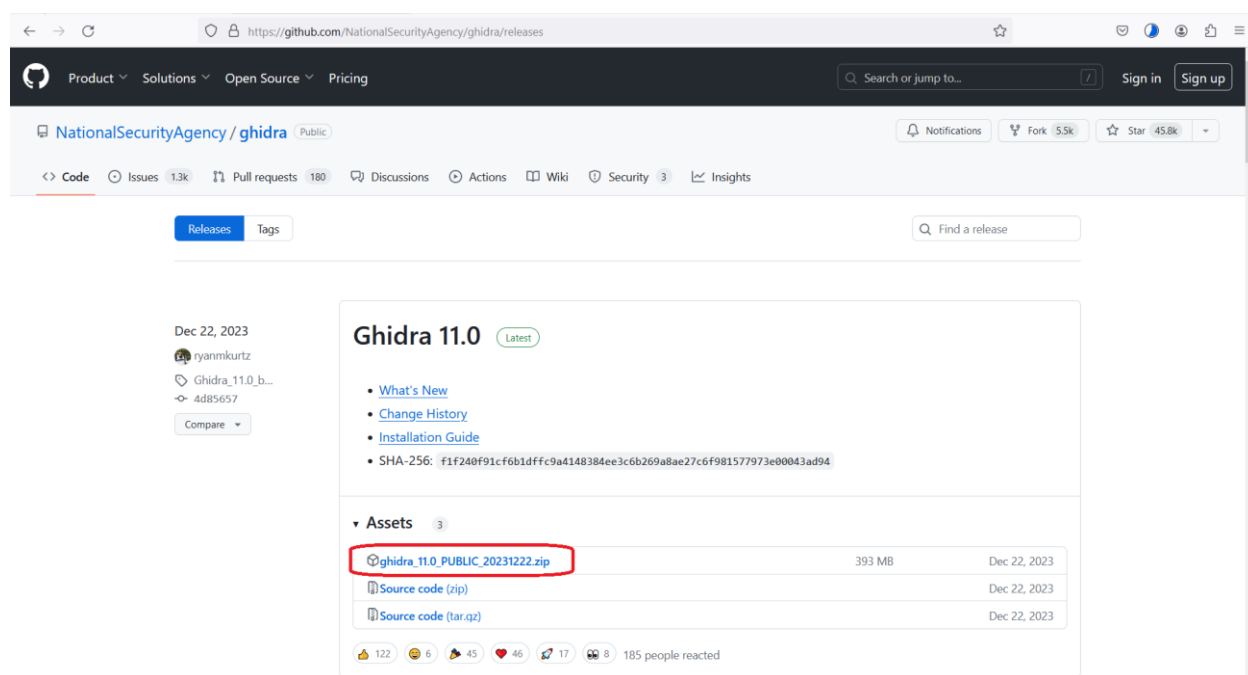


Рисунок 3.1 – Необходимый файл

После загрузки необходимо распаковать архив и перейти в папку с программой. Следующим шагом будет установка комплекта разработчика

приложений на языке Java JDK [8], если он ещё не установлен. Для этого необходимо загрузить файл, указанный на Рисунке 3.2. После загрузки распаковать архив в удобную директорию. После этого появится возможность работы с GHIDRA, но перед этим необходимо указать местонахождение JDK. Для этого будем использовать переменные среды. Для начала необходимо перейти в дополнительные параметры системы. Сделать это можно, нажав правой кнопкой мыши (ПКМ) по кнопке «Пуск», выбрав пункт «Система», как указано на Рисунке 3.3, и перейдя в соответствующий раздел, указанный на Рисунке 3.4.

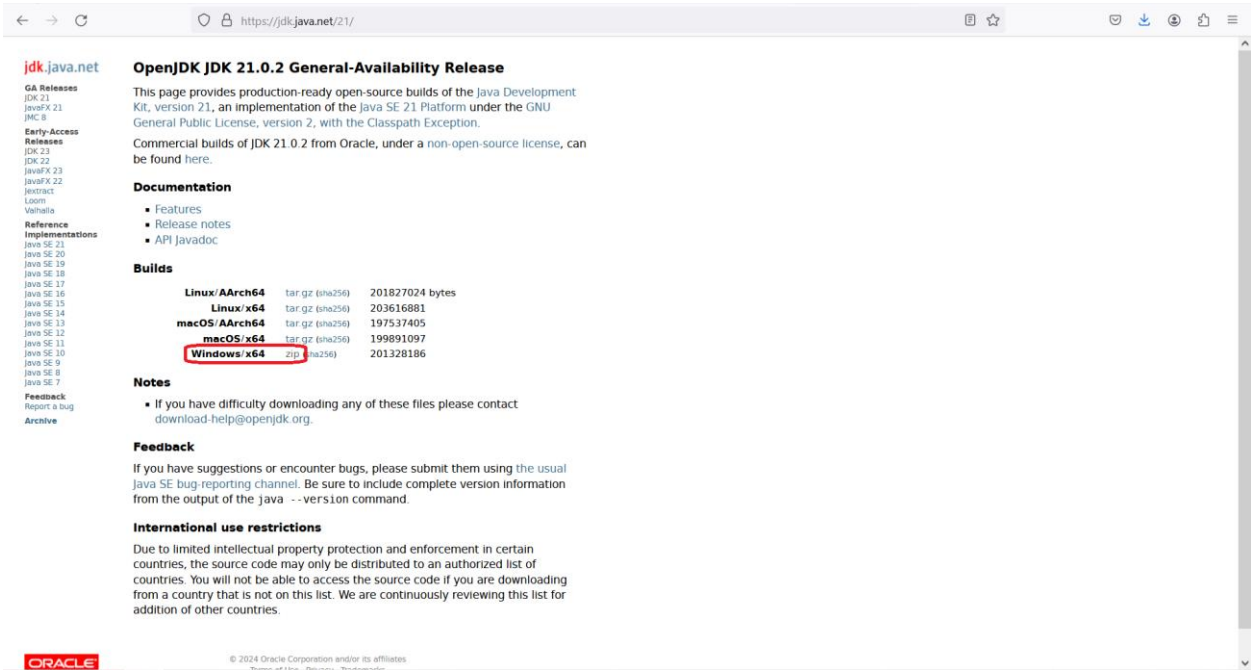


Рисунок 3.2 – Страница загрузки

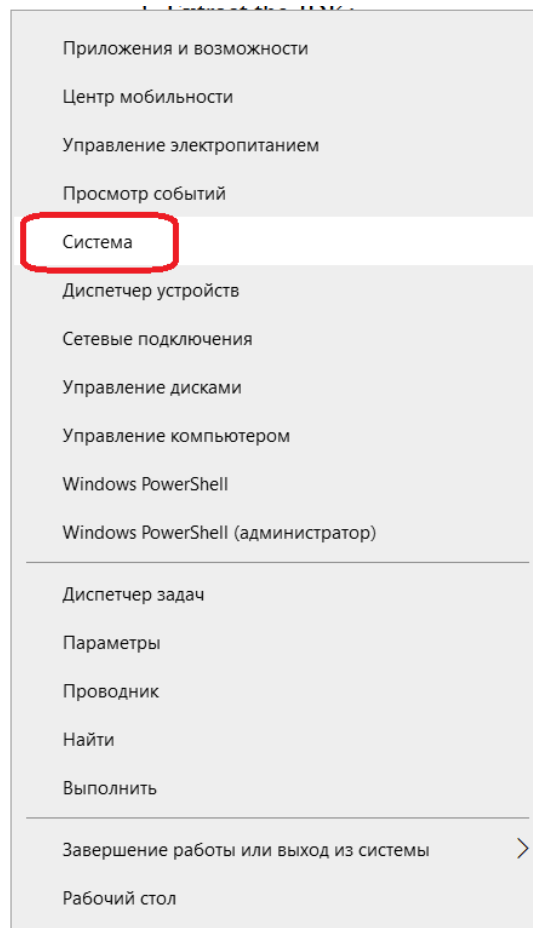


Рисунок 3.3 – Выбор подменю кнопки «Пуск»

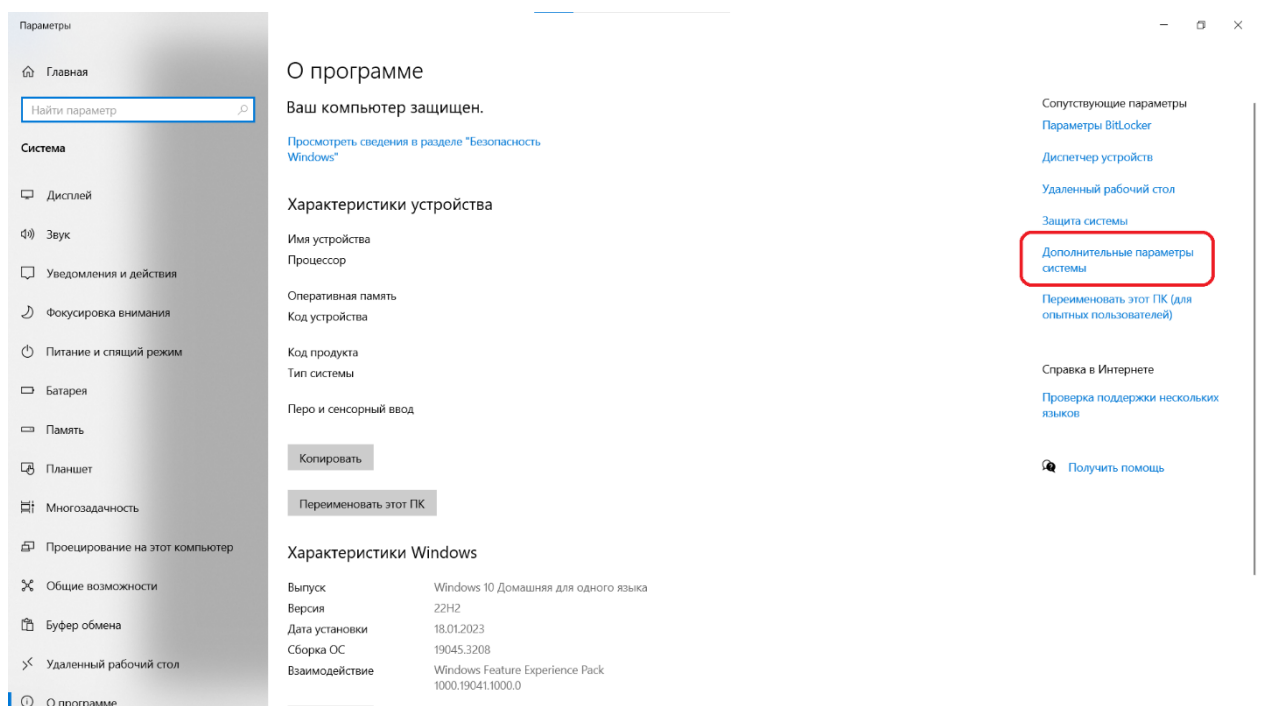


Рисунок 3.4 – Расположение меню дополнительных параметров

Далее осуществим выбор меню переменных среды, как показано на Рисунке 3.5.

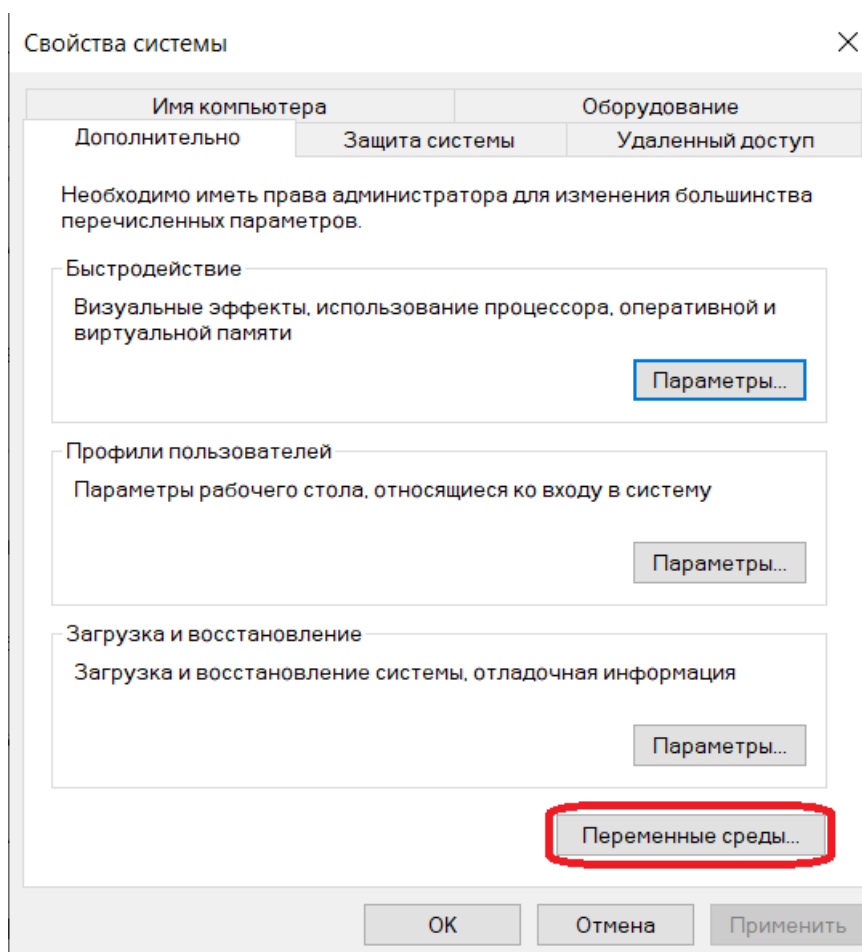


Рисунок 3.5 – Подменю «Переменные среды»

Теперь необходимо выбрать внизу окна переменную Path, после чего нажать кнопку «Изменить». Процесс на Рисунке 3.6.

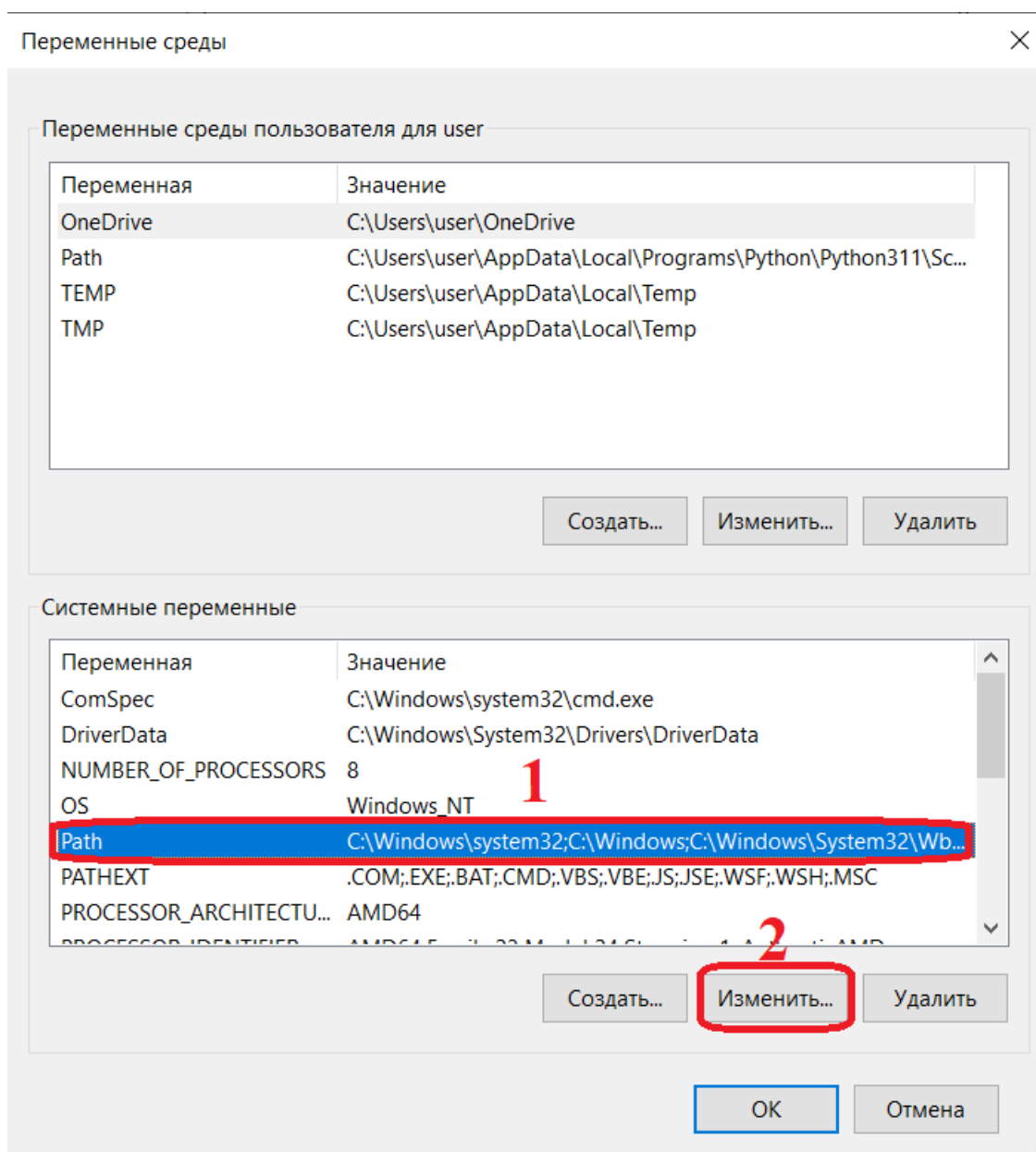


Рисунок 3.6 – Необходимая переменная среды

Далее, в появившемся окне требуется нажать кнопку «Создать». Появится возможность добавить строку, в которую необходимо вписать путь к распакованному архиву JDK, а точнее, вложенной в него папке bin/. Затем необходимо нажать кнопку клавиатуры «Enter», после – кнопку «ОК». Процесс формализован на Рисунке 3.7.

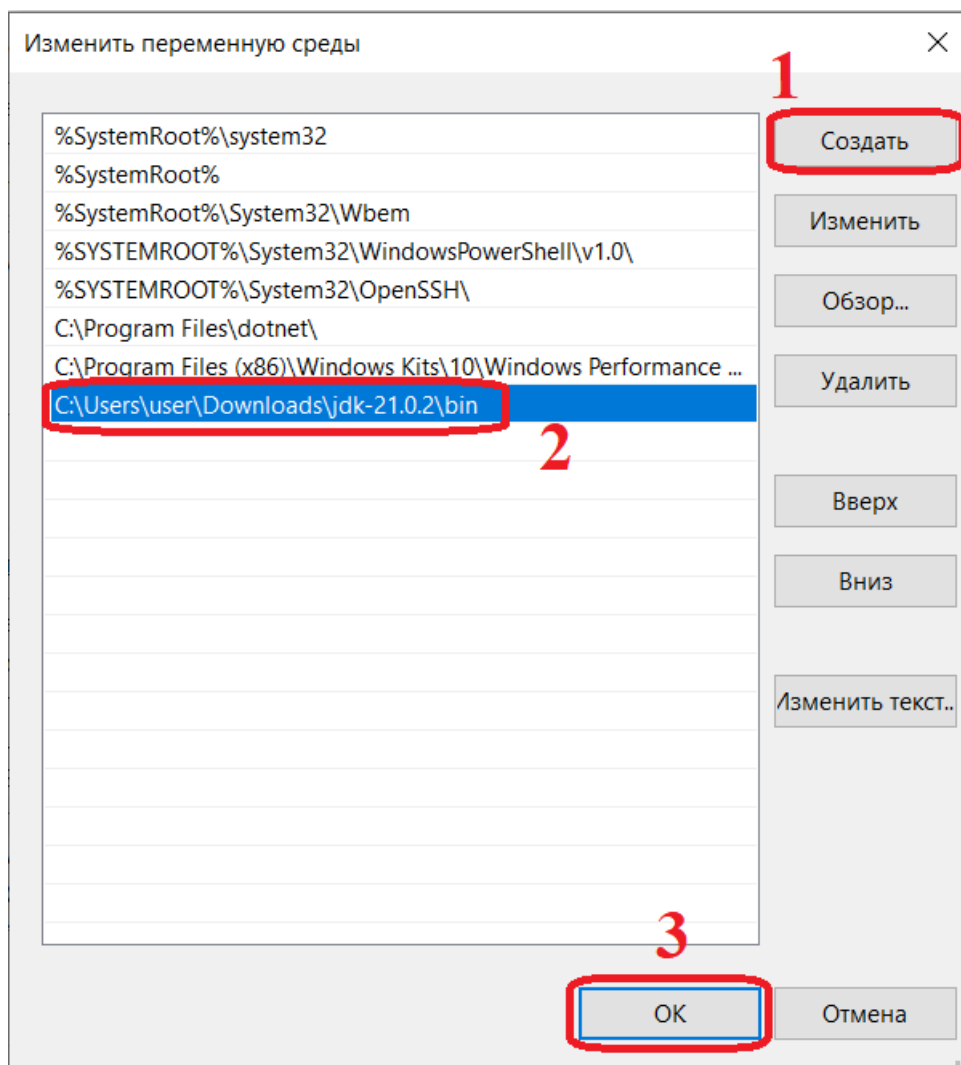


Рисунок 3.7 – Процесс добавления переменной среды

Когда предыдущие этапы выполнены, можно начать работу с GHIDRA. Для этого необходимо перейти в папку с программой, после чего запустить файл ghidraRun.bat, как показано на Рисунке 3.8.

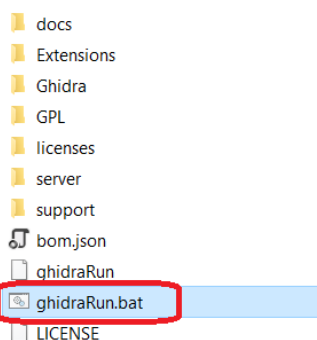


Рисунок 3.8 – Необходимый файл

После запуска необходимо прочесть соглашение и принять его, как показано на Рисунке 3.9.

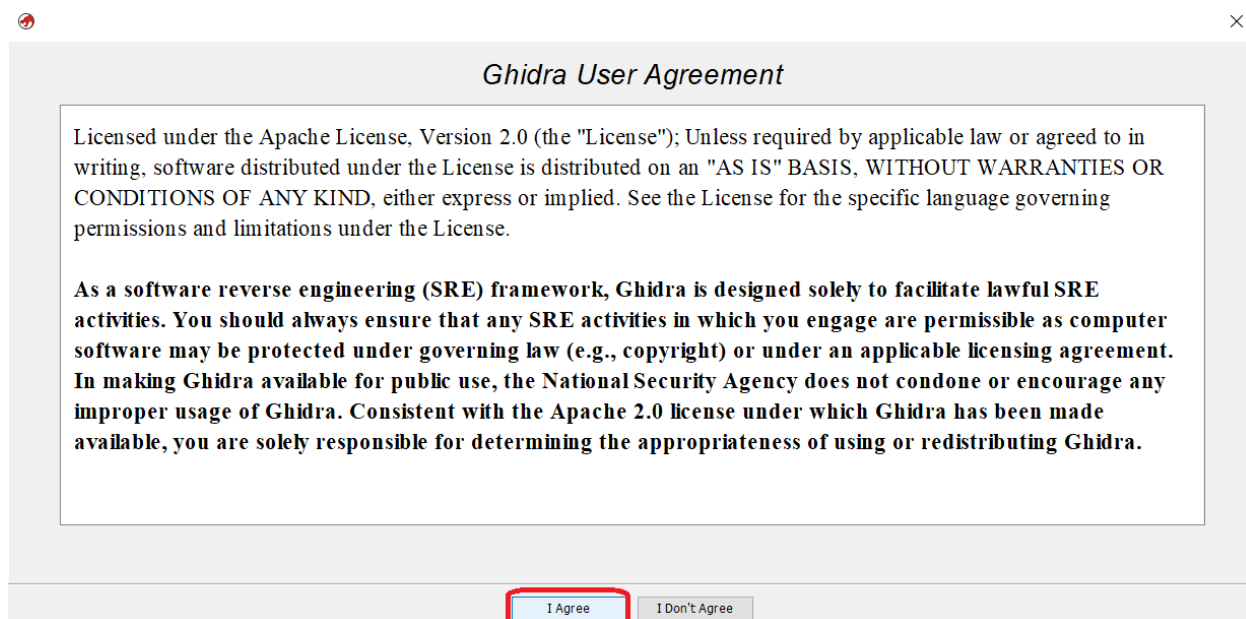


Рисунок 3.9 – Пользовательское соглашение

После принятия соглашения откроется интерфейс помощи, где можно подробно прочесть про функционал программы, и интерфейс выбора проекта.

В интерфейсе проекта необходимо создать новый проект, как показано на Рисунках 3.10-3.12. Для проекта рекомендуется выделить отдельную папку во избежание неразберихи с файлами. Для практики реверс-инжиниринга выберем файл [9] с того же сайта, что и программа [5] из предыдущего раздела. Получим один файл, который при запуске просит ввести нас пароль. Произведём ввод наугад. Как видим (Рисунок 3.13), программа не приняла пароль.

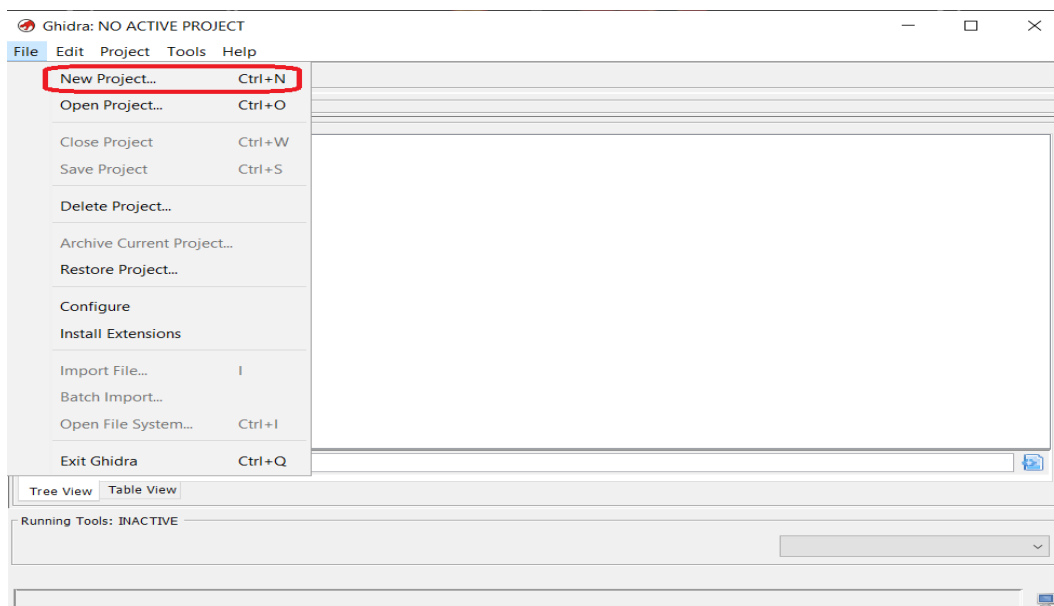


Рисунок 3.10 – Меню создания проекта

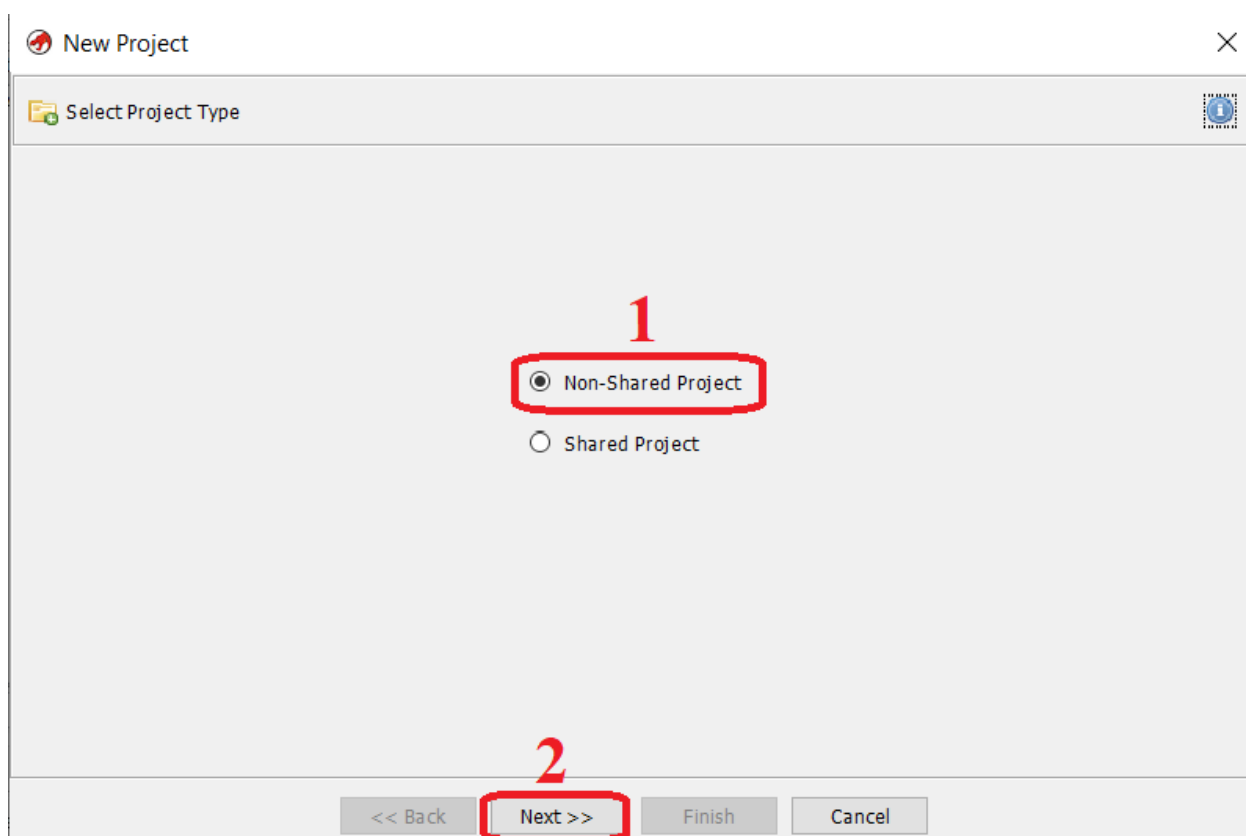


Рисунок 3.11 – Меню выбора режима (одионочный, обшй)

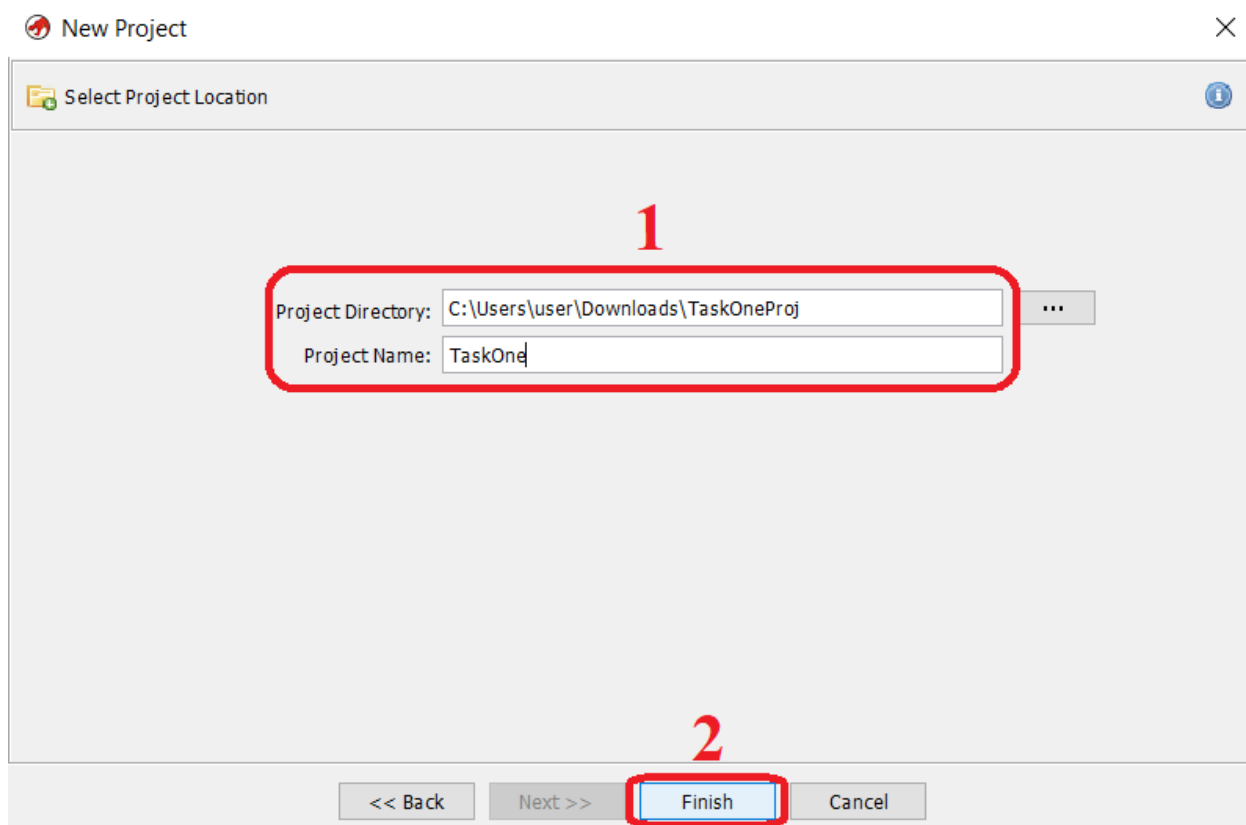


Рисунок 3.12 – Выбор директории проекта

```
C:\Users\user\Downloads>task.exe
Please~e, can~n~n you writew pas~s~sword <3 :
SomeMagickPassword
N~n~nop, you~u~u fail~l~led :(((
```

Рисунок 3.13 – Результат ввода неверного пароля

Откроем этот файл в проекте. Для этого в окне GHIDRA выберем пункт меню «Import file», после выберем нашу программу. Расположение указано на Рисунке 3.14.

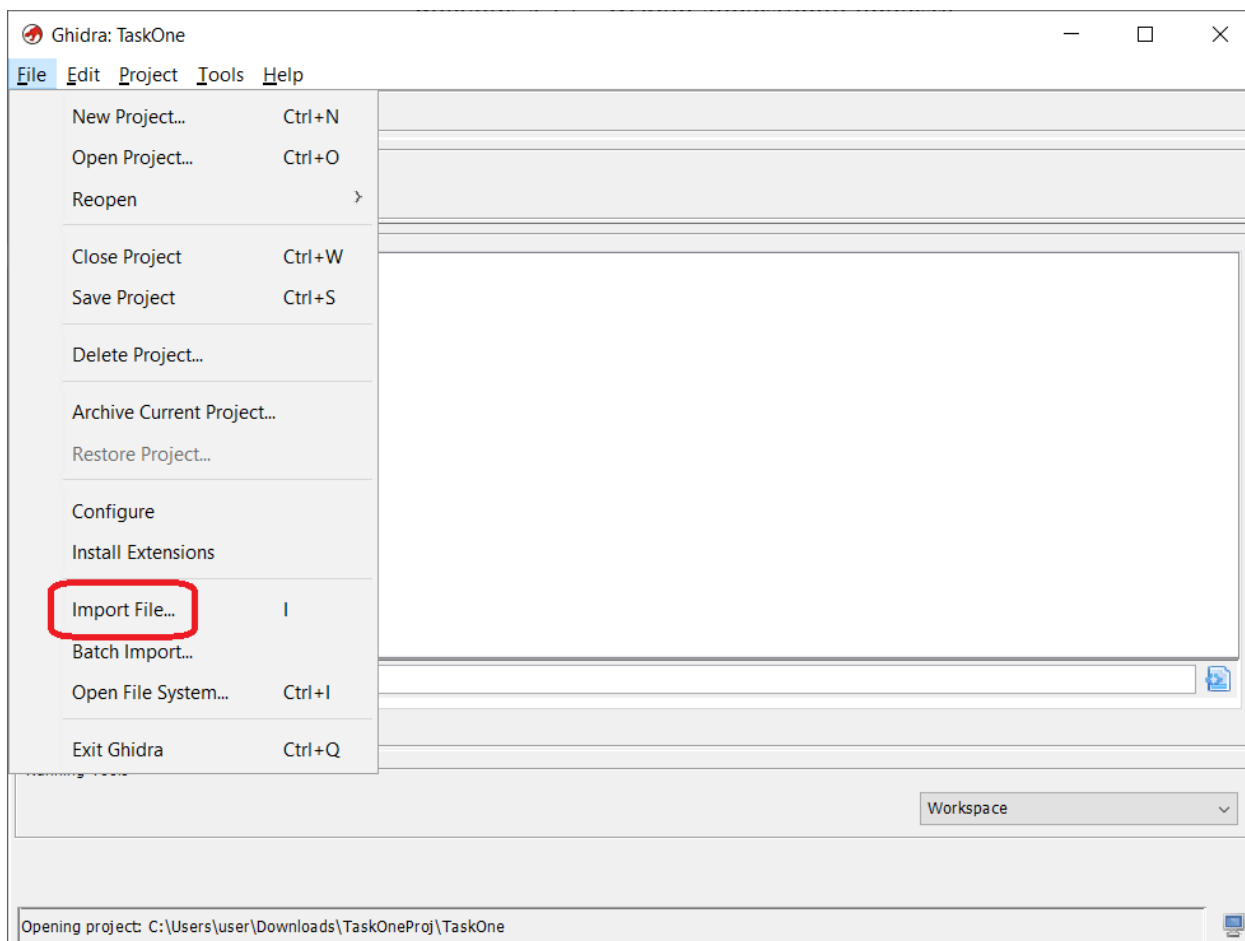


Рисунок 3.14 – Меню импорта файла в проект

Во время импорта файла настройки для анализа рекомендуется оставить по умолчанию. После импорта откроется окно с результатами анализа, где можно ознакомиться с различной информацией, касающейся загруженного файла. Это отображено на Рисунке 3.15. Для дальнейшего анализа необходимо запустить основной инструмент GHIDRA – анализ кода. Для этого требуется нажать на иконку зелёного дракона в папке с проектом (Рисунок 3.16).

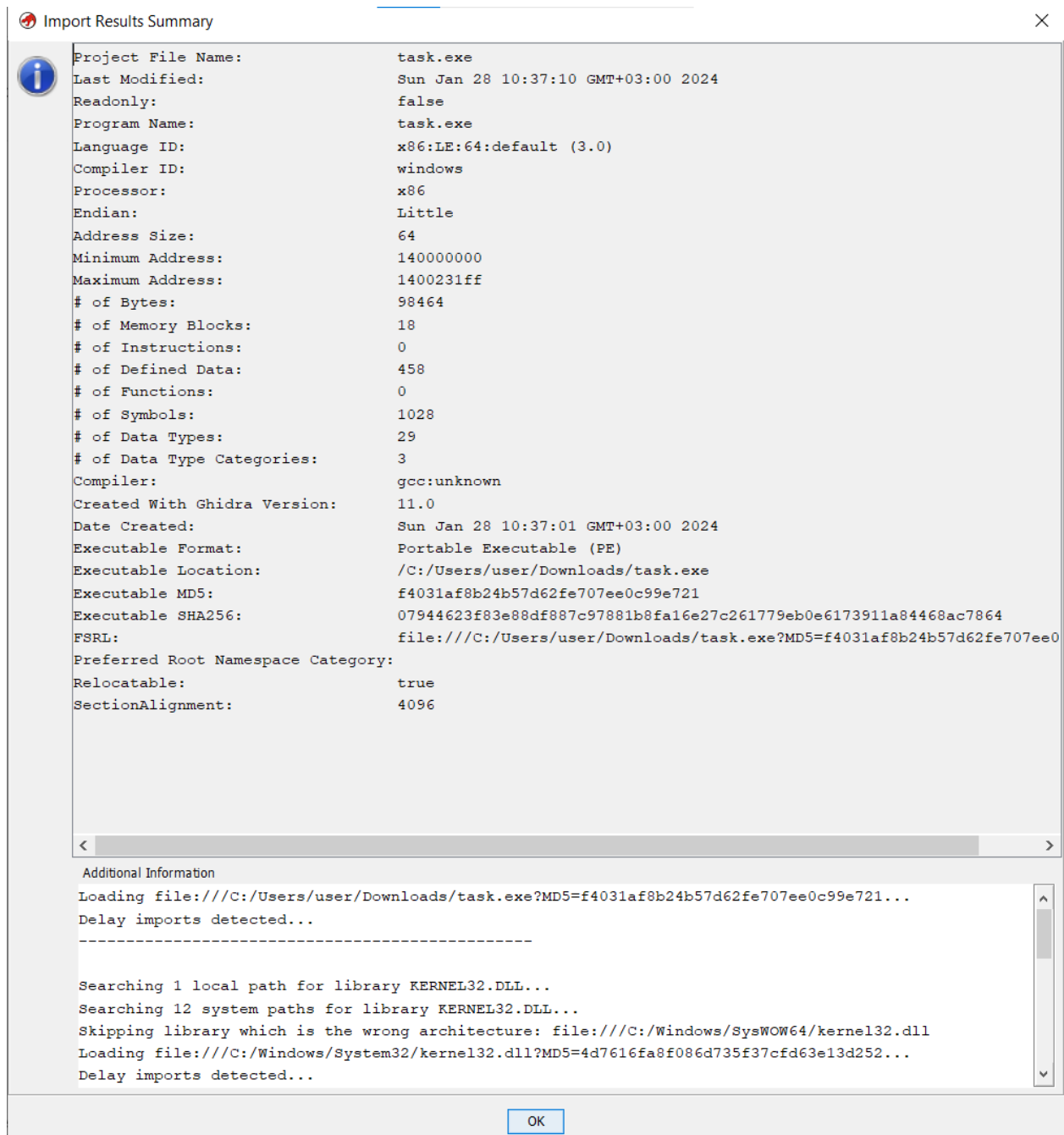


Рисунок 3.15 – Основная информация о файле

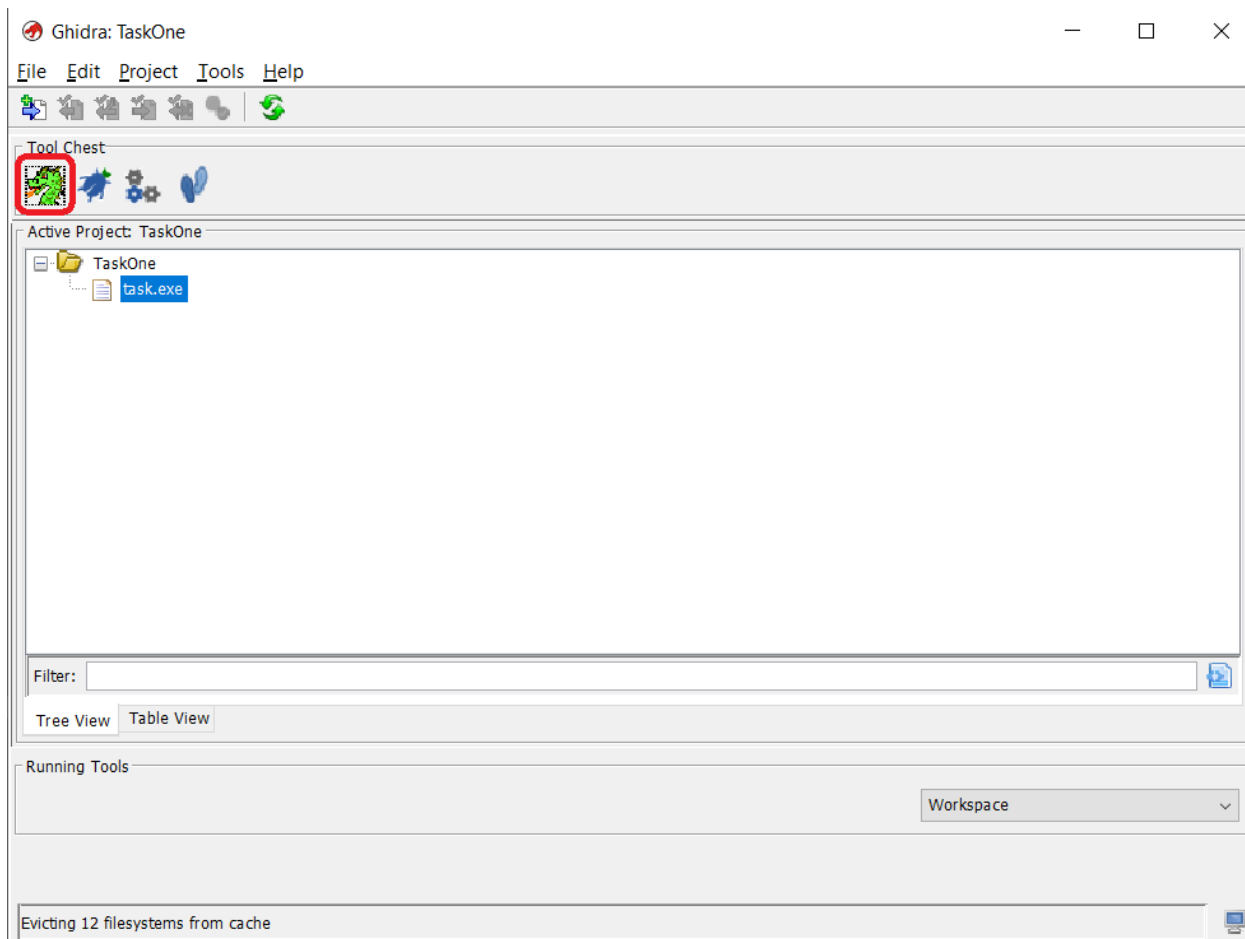


Рисунок 3.16 – Запуск анализа кода

Далее необходимо открыть файл. Для этого выберем соответствующий файл в пункте на Рисунке 3.17.

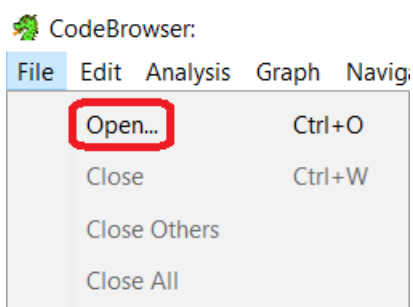


Рисунок 3.17 – Выбор файла для анализа

После выбора программа сообщит, что файл ещё не был проанализирован. Чтобы проанализировать файл, нажмите на кнопку «Yes», как показано на Рисунке 3.18. Затем появится окно, в котором можно

выбрать, что конкретно необходимо проанализировать в файле. Есть три основных блока: 1 – общий список опций, 2 – описание, 3 – выбор дополнительных настроек. Меню показано на Рисунке 3.19. Для начала оставим все пункты так, как есть.

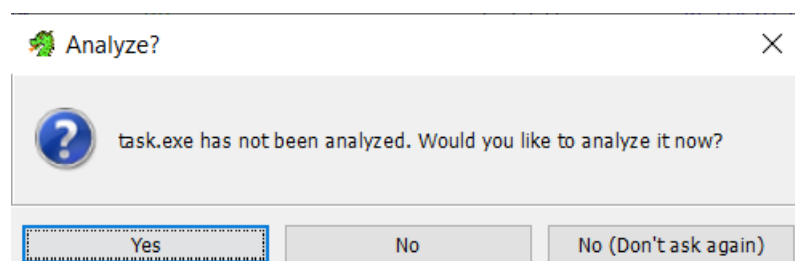


Рисунок 3.18 – Решение об анализе файла

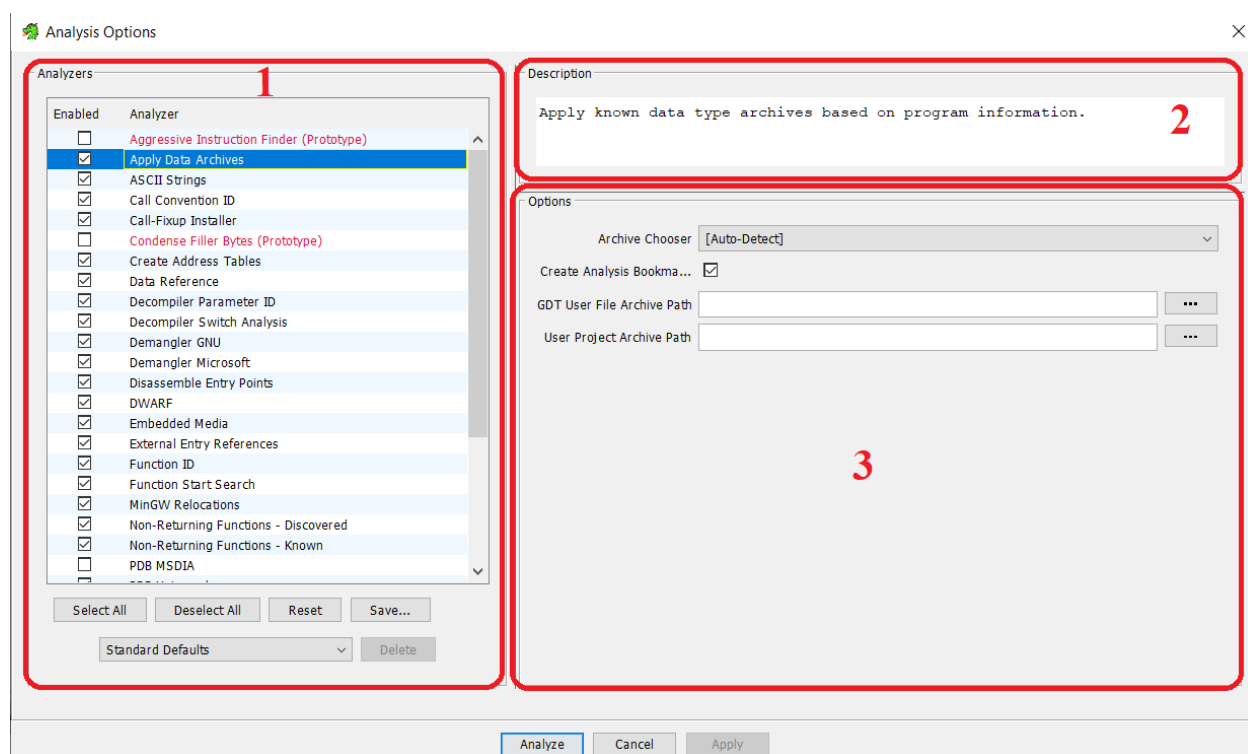


Рисунок 3.19 – Опции анализа

После того, как завершится анализ, откроется основное окно с инструментами отладки. Оно показано на Рисунке 3.20 и содержит следующие элементы: 1 – основная панель инструментов, 2 – основные блоки данных программы, 3 – дерево всех вхождений строк и быстрое перемещение

по ним, 4 – управление типами данных из программы, 5 – меню листинга с дизассемблированным кодом, 6 – меню с декомпилированным кодом, 7 – КОНСОЛЬ.

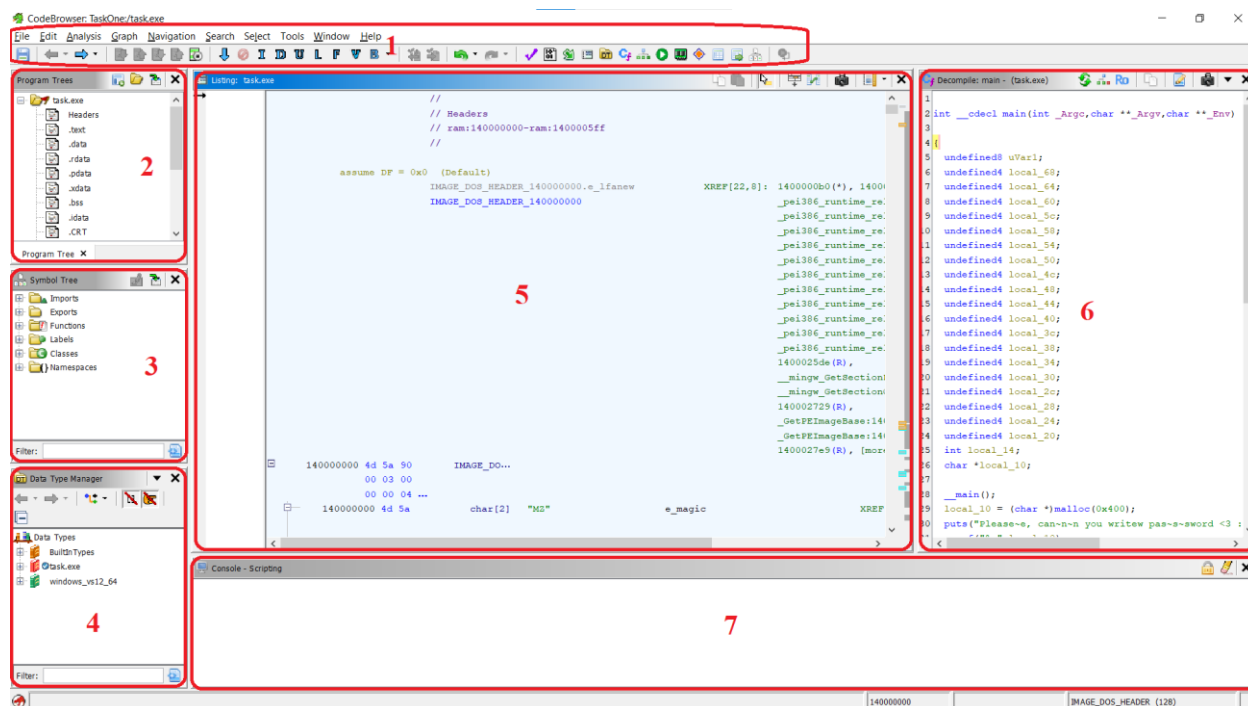


Рисунок 3.20 – Основной интерфейс после анализа

Попробуем воспользоваться некоторыми меню для нахождения пароля. Для начала выберем поиск строк. Для этого, как показано на Рисунке 3.21, выберем пункт и позднее нажмём «Search», оставив настройки поиска по умолчанию.

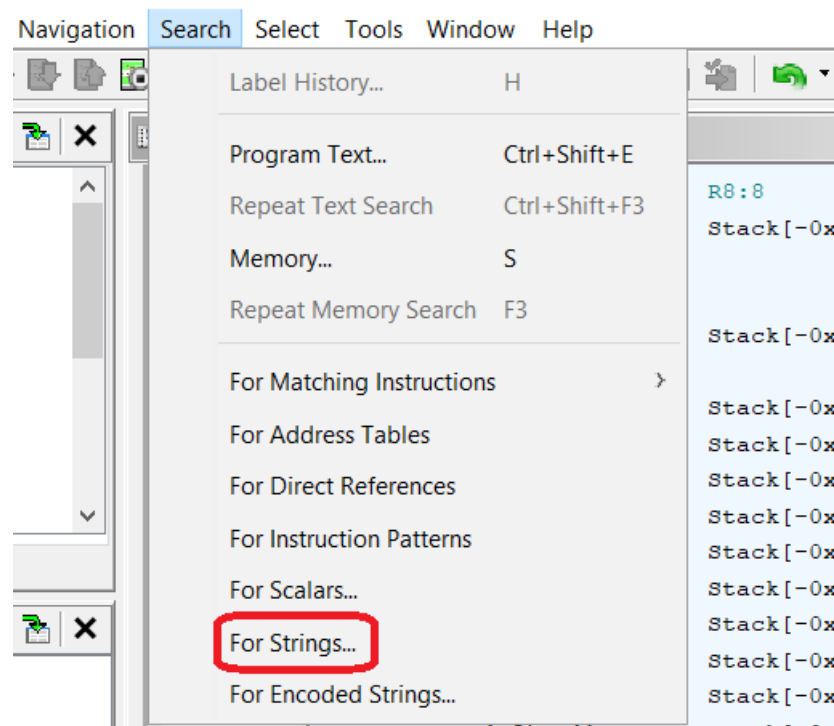


Рисунок 3.21 – Поиск строк в программе

После выполненного поиска получим окно, отображённое на Рисунке 3.22, в котором легко заметить несколько интересных строк. Перейдём по адресу первой из интересующих нас строк, дважды нажав на неё. Как можно видеть на Рисунке 3.23, данная строка ссылается на 3 других адреса: неизвестный, и два адреса из функции `main`. Перейдём по первому адресу для функции `main`, также дважды нажав по нему.

140001674	e8 e7 01	CALL	__main
	00 00		
140001679	b9 00 04	MOV	_Argc, 0x400
	00 00		
14000167e	e8 05 15	CALL	malloc
	01 00		
140001683	48 89 45 f8	MOV	qword ptr [RBP + local_10], RAX
140001687	48 8d 05	LEA	RAX, [s_Please~e, _can~n~n_you_writew_pas_140014.
	72 29 01 00		
14000168e	48 89 c1	MOV	Argc=>s_Please~e, _can~n~n_you_writew_pas_1400.
140001691	e8 0a 15	CALL	puts
	01 00		
140001696	48 8b 45 f8	MOV	RAX, qword ptr [RBP + local_10]
14000169a	48 89 c2	MOV	_Argv, RAX
14000169d	48 8d 05	LEA	RAX, [DAT_14001402e]
	8a 29 01 00		
1400016a4	48 89 c1	MOV	_Argc=>DAT_14001402e, RAX
1400016a7	e8 84 fe	CALL	scanf

Рисунок 3.24 – Функция, по которой можно перейти

Просмотрим декомпилированный код на Рисунке 3.25. Здесь можно увидеть несколько интересных блоков. После вывода строки на экран с помощью функции puts (1) у пользователя запрашивается ввод в переменную local_10 функцией scanf (2). В следующем блоке (3) объявляется список постоянных значений. Затем, в блоке (4) происходит вызов функции str_pass, и перевод её результата в булевый вид (true, false). Наконец, последний блок (5) в зависимости от значения переменной в предыдущем блоке выполняет ветвление и выводит строки на экран, одну в случае успешной проверки, и другую – в противном случае. После этого программа завершает работу.

```

__main();
local_10 = (char *)malloc(0x400);
puts("Please~e, can~n~n you writew pas~s~sword <3 :");
scanf("%s", local_10);
local_68 = 0x36617ab5;
local_64 = 0x36617a9e;
local_60 = 0x36617ab0;
local_5c = 0x36617a95;
local_58 = 0x36617ab6;
local_54 = 0x36617a9b;
local_50 = 0x36617abb;
local_4c = 0x36617a84;
local_48 = 0x36617ac1;
local_44 = 0x36617ace;
local_40 = 0x36617ac1;
local_3c = 0x36617ace;
local_38 = 0x36617ac6;
local_34 = 0x36617ac4;
local_30 = 0x36617ac4;
local_2c = 0x36617ac5;
local_28 = 0x36617ac5;
local_24 = 0x36617ac5;
local_20 = 0x36617acf;
uVar1 = cmp_pass(local_10, (longlong) &local_68);
local_14 = (int)uVar1;
if (local_14 == 0) {
    puts("N~n~nop, you~u~u fail~l~led :((");
}
else {
    puts("Ye~e~eah, you~u, righ~h~ht ;)))");
    puts("there~e codesw from nuclearw bombs~s: ");
    printf("%d\t%d\n%d\t%d", 0x26a1f, 0xe9f8f, 0x7931c, 0x62f65);
}

```

Рисунок 3.25 – Блок функции main

Посмотрим детальнее функцию `str_pass`. В неё передаются введённые пользователем данные и адрес первой переменной блока (2). Перейдём внутрь функции, что отражено на Рисунке 3.26. Сначала данная функция подсчитывает число символов в введённой пользователем строке, и если оно не равно 19 (0x13 в шестнадцатеричной системе), то возвращается 0. В противном случае выполняется цикл, в котором возвращается 0 в случае несоответствия. Следует обратить внимание, что при отсутствии изменения настроек оформления один символ из строки с условием переносится на следующую строку.

```

undefined8 cmp_pass(char *param_1,longlong param_2)
{
    size_t sVar1;
    undefined8 uVar2;
    uint local_c;

    sVar1 = strlen(param_1);
    if (sVar1 == 0x13) {
        for (local_c = 0; local_c < 0x13; local_c = local_c + 1) {
            if (param_1[local_c] != (byte)((byte)*(undefined4*)(param_2 + (ulonglong)local_c * 4) ^ 0xf7))
                return 0;
        }
        uVar2 = 1;
    }
    else {
        uVar2 = 0;
    }
    return uVar2;
}

```

Рисунок 3.26 – Функция cmp_pass

Подробно разберём условие. В нём выполняется проверка на неравенство символа, введенного пользователем, некому символу из памяти. Этот символ получается, как байт памяти со смещением по циклу, с применённой к нему операцией XOR (^). Затем полученное преобразуется в байт и подаётся на вход оператора сравнения. Разберём подробнее этот механизм. Так как мы имеем порядок записи little endian, что можно было заметить на Рисунке 3.15, то байты из блока 3 на Рисунке 3.25 мы должны брать в обратном порядке, с конца блока.

Заметим, что значение параметра цикла на каждой итерации умножается на 4, что означает смещение на 4 байта вверх по памяти (в данном случае вниз, из-за обратной записи). Возьмём первые данные. Это будет строка 0x366617AB5. Программа применяет операцию XOR с F7 лишь к последнему байту. Результат представлен на Рисунке 3.27. Получим код символа ASCII 0x42, и по таблице соответствия [10], фрагмент которой представлен на Рисунке 3.28, найдём нужный символ. В данном случае, это английская заглавная «В». Выполнив смещение на 4 байта, получим

следующие исходные данные: 9E. Снова проделав действие XOR F7, получим 0x69, что соответствует английскому символу «i». Прделаем операцию до конца. В результате получим строку «BiGbAlLs69691332228». Попробуем ввести её в нашу программу в качестве пароля. На Рисунке 3.29 отражён результат ввода. Как видим, мы успешно обнаружили пароль.

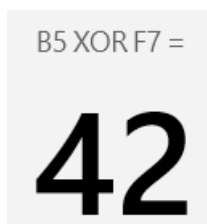


Рисунок 3.27 – Операция XOR

HEX	DEC	Symbol
0x40	64	@
0x41	65	A
0x42	66	B
0x43	67	C
0x44	68	D
0x45	69	E
0x46	70	F

Рисунок 3.28 – Таблица соответствия

```
C:\Users\user\Downloads>task.exe
Please~e, can~n~n you writew pas~s~sword <3 :
BiGbAlLs69691332228
Ye~e~eah, you~u, righ~h~ht ;)))
there~e codesw from nuclearw bombs~s:
158239 958351
496412 405349
```

Рисунок 3.29 – Результат с «найденным» паролем

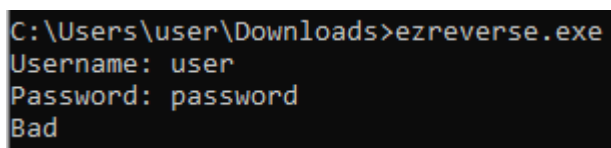
В данном разделе рассмотрены базовые инструменты, предоставляемые GHIDRA для отладки программ. Следует помнить, что данный инструмент является легко расширяемым. Это позволяет значительно увеличить число доступных инструментов, которые и без дополнительных модулей имеют необходимый для отладки функционал. Многое было оставлено без внимания с целью самостоятельного изучения данного инструмента. Итак, данное ПО позволяет при должном уровне знаний обходить защиту вредоносных приложений и исследовать их на скрытые возможности и изъяны логики. Учитывая то, что GHIDRA является бесплатным решением с открытым исходным кодом, и в силу сказанного, данная среда может полноценно заменить коммерческие решения.

3.2. ПРИМЕНЕНИЕ ИНСТРУМЕНТА X64DBG

Инструмент x64dbg является программой для отладки. Он является решением с открытым исходным кодом. Данная программа подходит для отладки приложений на платформе Windows x32 и Windows x64. Отладчик имеет понятный интерфейс и множество функций, среди которых имеется поддержка плагинов и скриптовых функций.

Подробное описание возможностей программы, равно как и ссылки для её установки, находятся на домашней странице (<https://x64dbg.com/>). Установка производится путём скачивания и распаковки архива с программой. В директории необходимо найти папку release, где предоставлена возможность выбора необходимой версии. Запуск x96dbg.exe приведёт к установке обеих версий на компьютере в виде приложений. Запустить версии программы можно без установки по отдельности. Для этого перейдите в необходимую директорию (x32, x64) и запустите в ней соответствующий файл (x32dbg.exe, x64dbg.exe). После этого откроется сама программа.

Для отладки выберем программу [11] с сайта crackmes.one. Запустим её и введём произвольные данные. Как видим, результат нашей попытки войти отрицательный (Рисунок 3.31).



```
C:\Users\user\Downloads>ezreverse.exe
Username: user
Password: password
Bad
```

Рисунок 3.31 – Результат попытки ввода пароля

Запустим отладку, выбрав в x64dbg подменю «Файл», а далее пункт меню «Открыть» (Рисунок 3.32). Открывается множество функциональных областей. Разберём их предназначение более подробно. Сами области отражены на Рисунке 3.33.

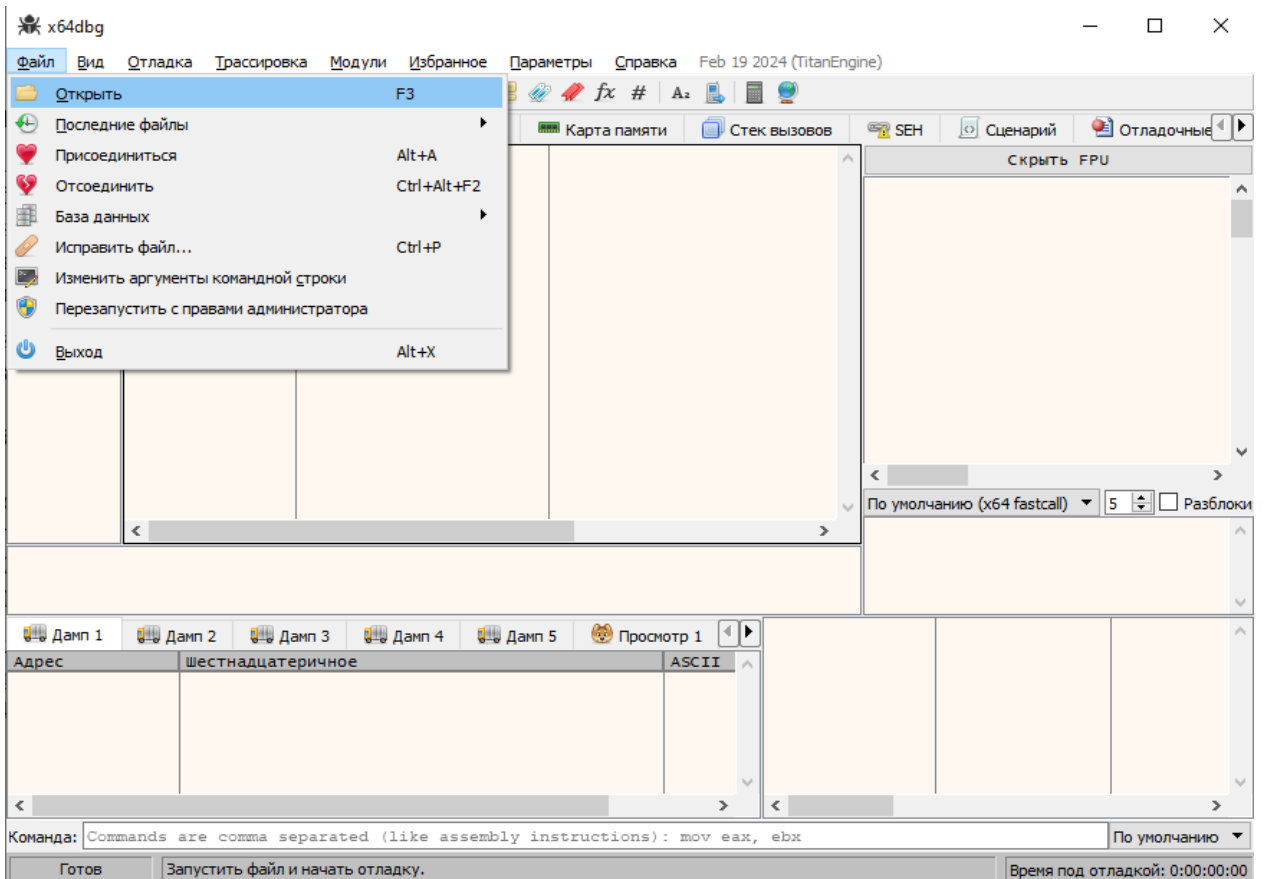


Рисунок 3.32 – Пункт меню для выбора файла

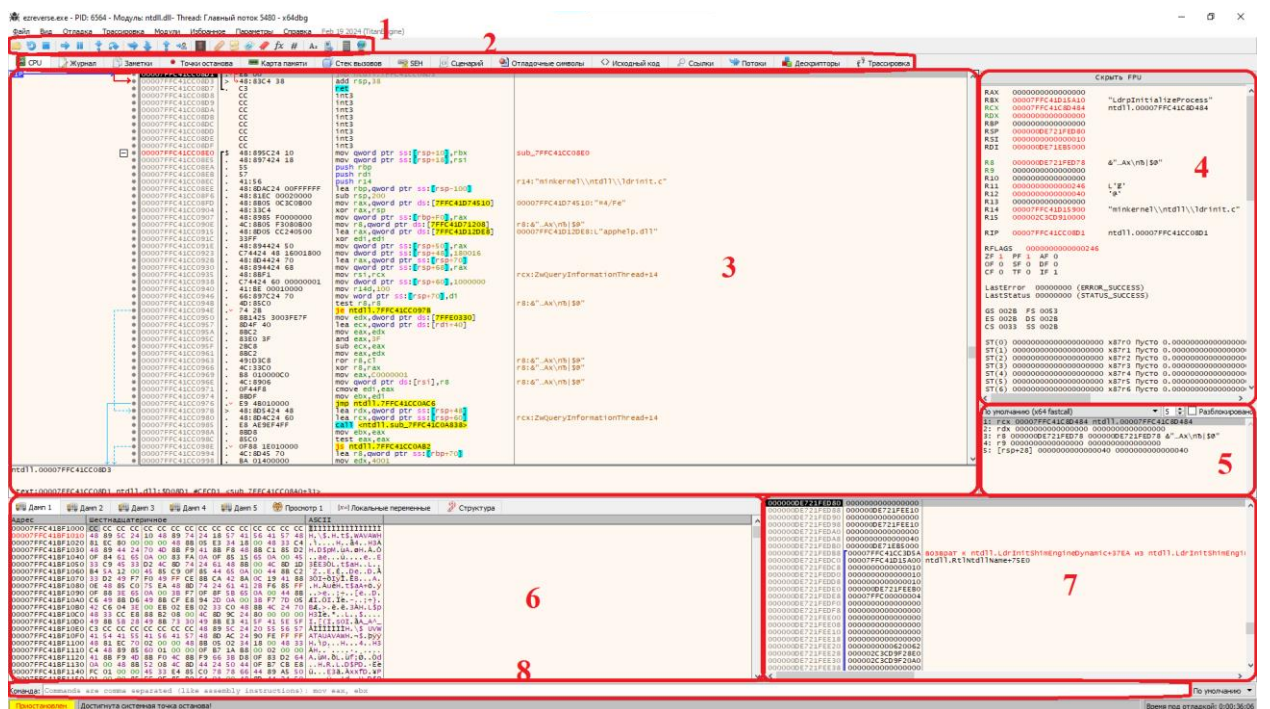


Рисунок 3.33 – Основное содержание программы

Первая область – элементы управления ходом программы и анализом. Вторая отвечает за открытие дополнительных окон программы. Третья – основная область, содержащая дизассемблированный код программы. Четвёртая содержит данные о содержимом регистров, а пятая – о содержимом стека. В шестой хранятся дампы памяти программы и прочие данные. Седьмой содержит информацию, а восьмой – командная строка, позволяющая ввести в программу разнообразные команды.

Программа начинает своё выполнение с инициализации системных данных. Чтобы понять, какой модуль исполняется в текущий момент, достаточно взглянуть на название программы. На Рисунке 3.34 видно, что сейчас идёт выполнение `ntdll.dll` – библиотеки, которая вызывает прерывания на системе с ядром NT. Чтобы выполнить модуль до следующего ключевого элемента, необходимо нажать соответствующую кнопку (Рисунок 3.35), либо клавишу F9. Информация в окне обновится на новую (Рисунок 3.36), а значит можно найти строковые вхождения для оптимизации процессе (Рисунок 3.37).

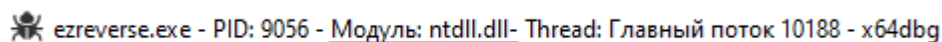
The image shows a debugger window with a title bar that reads "ezreverse.exe - PID: 9056 - Модуль: ntdll.dll - Thread: Главный поток 10188 - x64dbg". The text "Модуль: ntdll.dll" is underlined in red.

Рисунок 3.34 – Текущий исполняемый модуль

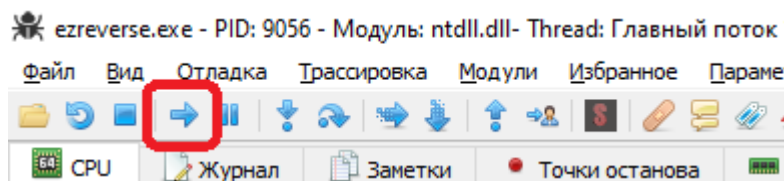


Рисунок 3.35 – Кнопка выполнения ключевого блока

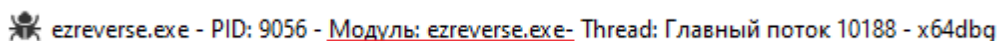
The image shows the same debugger window as before, but the title bar now reads "ezreverse.exe - PID: 9056 - Модуль: ezreverse.exe - Thread: Главный поток 10188 - x64dbg". The text "Модуль: ezreverse.exe" is underlined in red.

Рисунок 3.36 – Новый исполняемый модуль

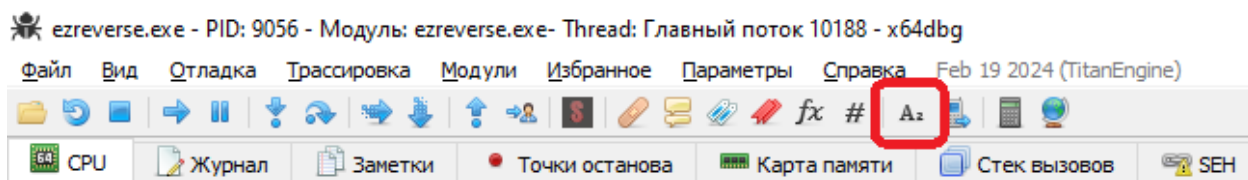


Рисунок 3.37 – Поиск строк в программе

После этого увидим, что открылся раздел ссылки. Здесь найдём интересные нас строки и адрес, по которому они используются (Рисунок 3.38). Перейдём по адресу строки «Password». Для этого нажмём ПКМ по адресу и выберем пункт меню (Рисунок 3.39).



Рисунок 3.38 – Строковые вхождения

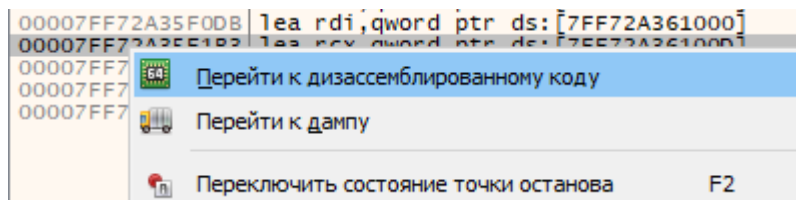


Рисунок 3.39 – Переход по адресу строкового вхождения

После перехода поставим точку останова по произвольному адресу чуть дальше строки. Для этого нажмём левую кнопку мыши на какой-либо строке, после чего нажмём F2. Адрес строки окрасится в красный цвет (Рисунок 3.40).

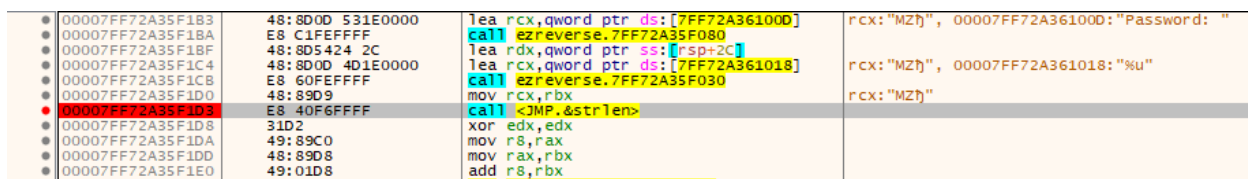


Рисунок 3.40 – Выставление точки останова

После этого выполним несколько раз программу, до тех пор, пока она не попросит нас ввести имя и пароль. Введём произвольные значения: 123 в качестве имени, и 321 в качестве пароля. Затем снова перейдём к дизассемблированному виду. Рассмотрим назначение действий в цикле на Рисунке 3.41.

call <JMP.&strlen>	Вычисление длины пароля
xor edx,edx	Предварительная очистка
mov r8,rax	Перемещаем длину в R8
mov rax,rbx	Копируем имя в RAX
add r8,rbx	Добавляем счётчик к строке
jmp ezreverse.7FF72A35F1F1	Прыгаем на первое сравнение *
nop dword ptr ds:[rax],eax	
movsx ecx,byte ptr ds:[rax]	# Записываем цифру по порядку в ECX
add rax,1	Обновляем счётчик
add edx,ecx	Добавляем ASCII код символа в EDX
cmp rax,r8	* Сравниваем, закончился ли счёт, и если нет, переходим #
jne ezreverse.7FF72A35F1E8	
cmp dword ptr ss:[rsp+2C],edx	Сравниваем наш пароль по адресу с полученной суммой, и если совпадают, переходим к выводу Good
je ezreverse.7FF72A35F216	
lea rcx,qword ptr ds:[7FF72A361021]	rcx:"123", 00007FF72A361021:"Bad\n"
call ezreverse.7FF72A35F080	Иначе выводим Bad
xor eax,eax	
add rsp,A8	
pop rbx	rbx:"123"
pop rsi	
pop rdi	rdi:"Username: "
pop rbp	
ret	
lea rcx,qword ptr ds:[7FF72A36101B]	rcx:"123", 00007FF72A36101B:"Good\n"
call ezreverse.7FF72A35F080	
jmp ezreverse.7FF72A35F208	

Рисунок 3.41 – Описание механизма проверки

Можно посмотреть переходы более детально, построив граф программы по нажатию кнопки G, вводу в поле 8 команды «graph», или через контекстное меню. Пример графа для данного участка кода предоставлен на Рисунке 3.42.

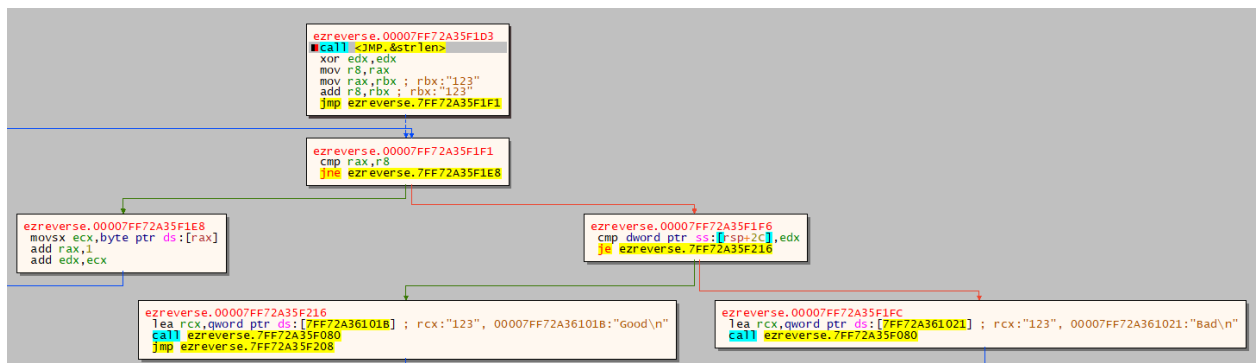


Рисунок 3.42 – Граф механизма проверки

Теперь, обладая данными знаниями, попробуем обойти проверку пароля. Для этого в качестве имени введём «usr», а в качестве пароля сумму ASCII-символов имени (u – 117, s – 115, r – 114), равную 346. Результат приведён на Рисунке 3.43.

```
Username: usr
Password: 346
Good
```

Рисунок 3.43 – Ввод полученных данных

Как видим, нам успешно удалось обойти проверку.

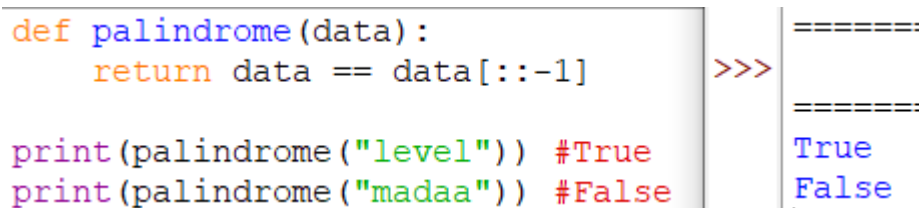
В результате знакомства с инструментом x64dbg можно утверждать, что в умелых руках это не только отладчик программ, но и полноценная среда для выполнения задач реверс-инжиниринга. Инструмент обладает богатым функционалом. x64dbg даёт возможность качественно и быстро анализировать программы. Благодаря своим преимуществам рекомендуется использовать этот отладчик для решения задач реверс-инжиниринга на системах под управлением Windows.

4. КОНВЕРТАЦИЯ ПРОГРАММНОГО КОДА С ОДНОГО ЯЗЫКА ПРОГРАММИРОВАНИЯ НА ДРУГОЙ

4.1. ПРИМЕНЕНИЕ ЯЗЫКОВОЙ МОДЕЛИ С ИСКУСТВЕННЫМ ИНТЕЛЛЕКТОМ В TELEGRAM

В настоящее время для конвертации одного ЯП в другой не существует универсального рабочего решения, за исключением написания кода с нуля. В данном разделе рассмотрим пример конвертации кода путём применения нейронных сетей. Конкретно в данном пункте воспользуемся ChatGPT v3.5 в мессенджере Telegram (<https://t.me/GPT4Telegrambot>).

Для того, чтобы перевести код, необходимо обратиться к боту с запросом «Перепиши код на C++» или похожим, при этом выбор ЯП не ограничен. Попробуем перевести простую программу, написанную на ЯП Python и выполняющую проверку на палиндромы, на ЯП C++. Сначала проверим работу программы на ЯП Python (Рисунок 4.1).



```
def palindrome(data):  
    return data == data[::-1]  
  
print(palindrome("level")) #True  
print(palindrome("madaa")) #False
```

The screenshot shows a code editor with Python code on the left and a terminal output on the right. The code defines a function `palindrome` that checks if a string is a palindrome. It then prints the results for "level" (True) and "madaa" (False). The terminal output shows the same results, with the prompt `>>>` and the output lines separated by blank lines.

Рисунок 4.1 – Запуск кода программы на ЯП Python

Как видим, код работает именно так, как предполагается. Теперь попросим чат-бота переписать наш код на ЯП C++, и проверим, насколько хорошо он справится с этим (Рисунок 4.2).

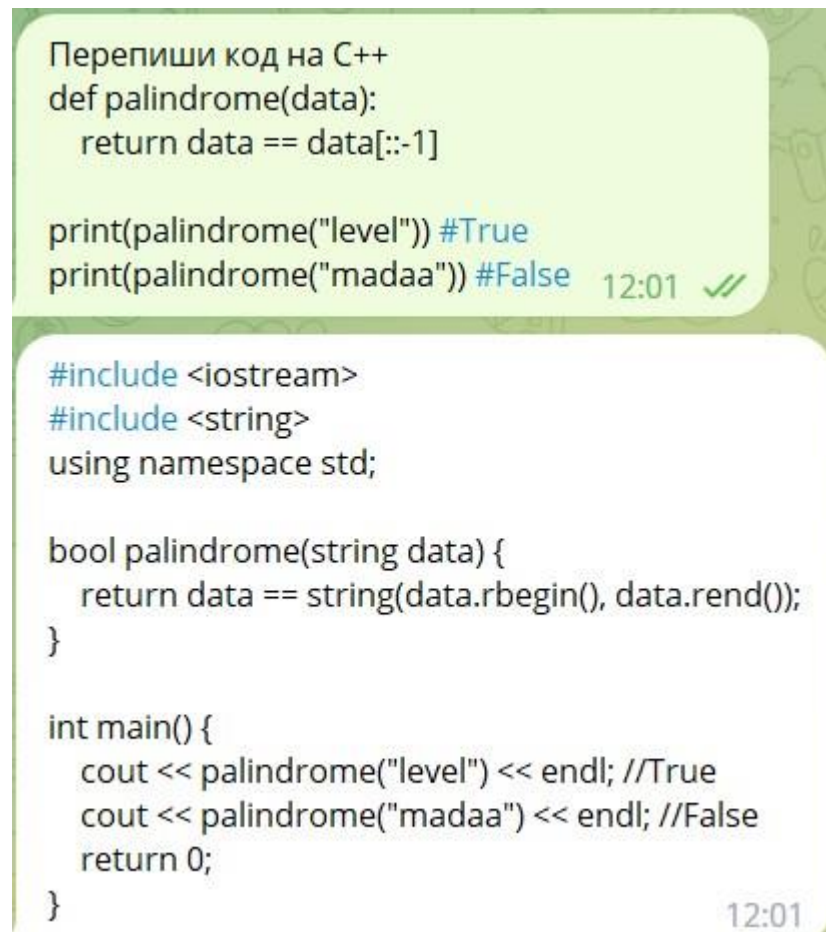


Рисунок 4.2 – Запрос боту на конвертацию кода

Выходной код загрузим в MVS, и там проверим его работоспособность (Рисунок 4.3).



Рисунок 4.3 – Загрузка и выполнение сгенерированного кода на ЯП C++

Как можно видеть, программа работает, как и задумывалось. Это означает, что нейронная сеть смогла правильно конвертировать код.

Этот пример является достаточно простым, поэтому попробуем ещё два: перевести программу среднего и высокого класса сложности. В качестве программы средней сложности возьмём консольную программу для игры в «виселицу», код приведён на Рисунке 4.4. Проверим её работу на ЯП Python (Рисунок 4.5)/

```
from random import shuffle
# Кол-во попыток.
turns = 10
print(f"Привет, Давай сыграем в виселицу! У тебя есть {turns} попыток!")
# Список слов, которые участвуют в игре.
wordList = ["geekflare", "awesome", "python", "magic"]
# Перемешиваем список.
shuffle(wordList)
# Берем последнее слово из списка.
word = wordList.pop()
guesses = ""
# Цикл, который будет работать, пока не останется попыток или не отгаданных букв.
while turns > 0:
    wrong = 0
    for char in word:
        if char in guesses:
            print(char, end=" ")
        else:
            print("_", end=" ")
            wrong += 1
    print("\n")
    if wrong == 0:
        print("Ты выиграл! :)")
        break
    print()
    guess = ""
    if len(guess) < 1:
        guess = input("Впиши букву и нажми enter: ")[0]
    if guess in guesses:
        print("Эта буква уже была!")
    guesses += guess
    if guess not in word:
        turns -= 1
        print("Упс! Ошибка")
        print(f"У тебя осталось {turns} попыток")
    if turns == 0:
        print("Ты проиграл! :(")
```

Рисунок 4.4 – Код «виселицы» на ЯП Python

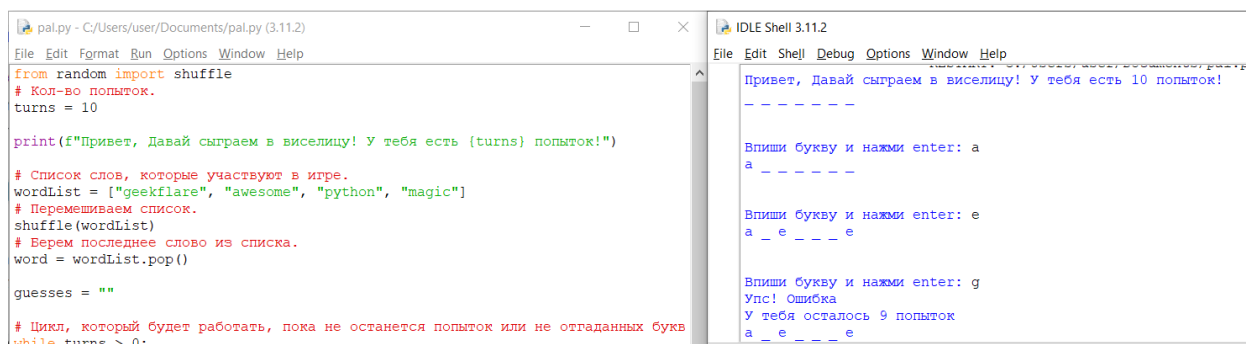


Рисунок 4.5 – Проверка работоспособности кода «виселицы»

Как видим, код успешно выполняется. Теперь попросим ChatGPT переписать его. Результат запуска переписанной программы на Рисунке 4.6.

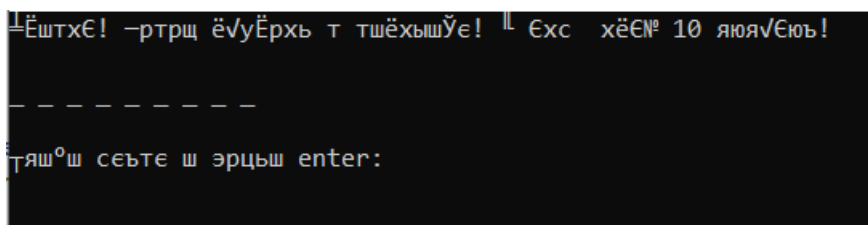


Рисунок 4.6 – Запущенная «виселица» на C++

Как можно видеть, появились первые проблемы. В данном случае проблема несерьёзная, т.к. просто отсутствует определение локализации. Попробуем исправить это с помощью соответствующей строки: «setlocale(LC_ALL, “ru”);». После того, как добавим данную строку, попробуем снова запустить программу (Рисунок 4.7).

```
Привет! Давай сыграем в виселицу! У тебя есть 10 попыток!  
  
- - - - -  
Впиши букву и нажми enter: е  
Упс! Ошибка  
У тебя осталось 9 попыток  
  
- - - - -  
Впиши букву и нажми enter: р  
  
р - - - - -  
Впиши букву и нажми enter: _
```

Рисунок 4.7 – Запуск отредактированной программы на C++

Как видим, тут уже потребовалось вносить правки в код, предоставленный сетью, для того чтобы получить желаемый результат.

Наконец, для «сложной» программы выберем ту, что работает с графикой. Для примера возьмём программу, которая рисует движущийся квадрат. Проверим её (Рисунок 4.8).

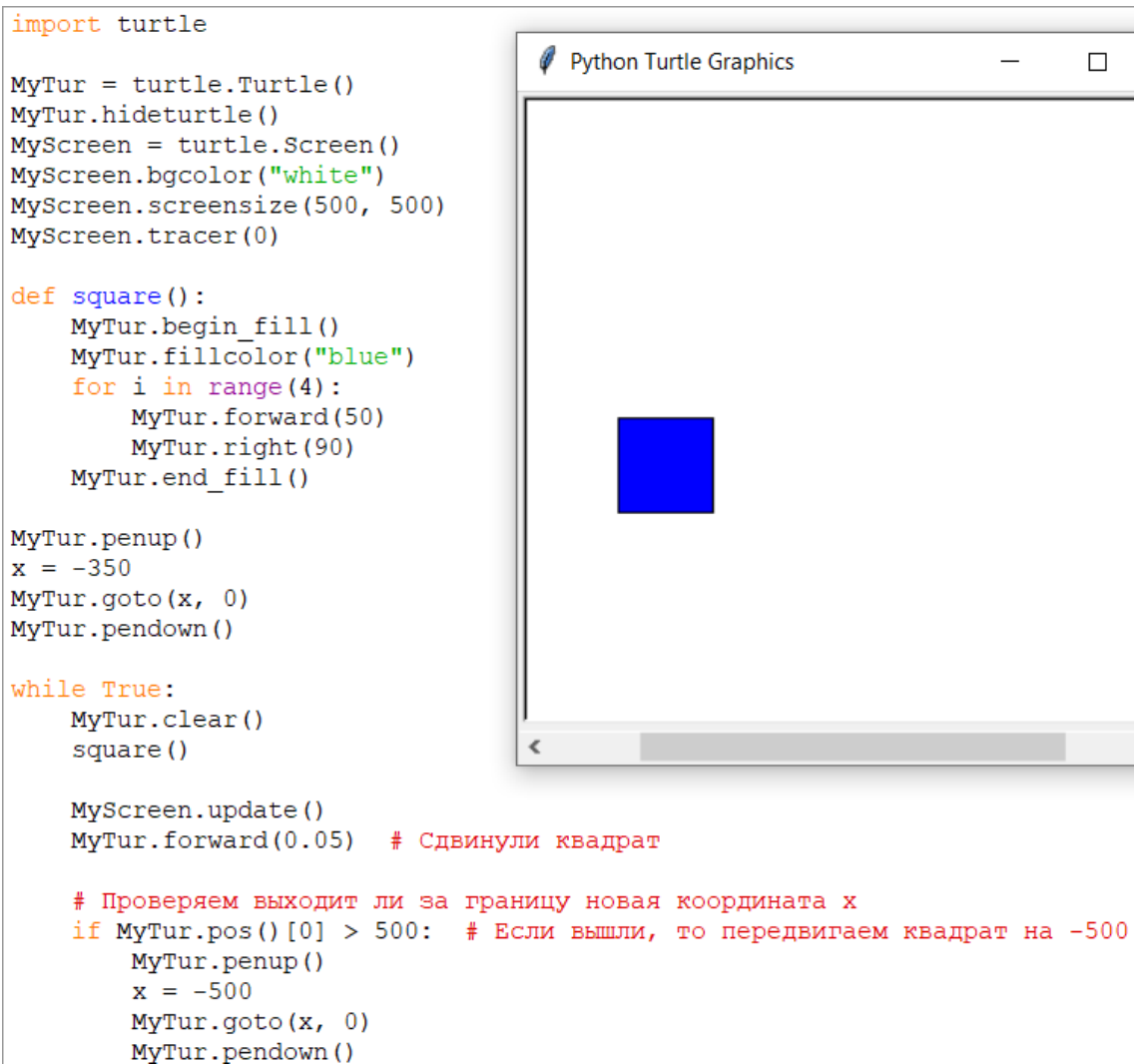


Рисунок 4.8 – Запуск «сложной» программы на ЯП Python

Попросим нейронную сеть снова переписать код. Переписанный код приведён на Рисунке 4.9.

Следует заметить, что при копировании кода компилятор, вероятнее всего, «пожалуется» на отсутствие необходимой библиотеки SFML. В данном примере уже произведена её установка и настройка решения для использования этой библиотеки по инструкции с сайта <https://habr.com/ru/articles/703500/> (дата обращения – 22.10.23). В рамках данного пособия не станем рассматривать подробную настройку, т.к. это не является целью данного раздела.

После копирования кода и настройки необходимых библиотек попробуем собрать решение. Как видим, эта попытка провалилась, и

компилятор выдал множество ошибок (Рисунок 4.10). Таким образом, перевод кода с ЯП Python на ЯП C++ провалился, а конечный код требует глубокого анализа, понимания ЯП и серьёзных доработок.

```
#include <iostream>
#include <cmath>
#include <SFML/Graphics.hpp>

int main()
{
    sf::RenderWindow window(sf::VideoMode(500, 500), "Moving Square");
    window.setFramerateLimit(60);

    sf::RectangleShape square(sf::Vector2f(50, 50));
    square.setFillColor(sf::Color::Blue);
    square.setPosition(-350, 0);

    while (window.isOpen())
    {
        sf::Event event;
        while (window.pollEvent(event))
        {
            if (event.type == sf::Event::Closed)
                window.close();
        }

        square.move(0.05, 0);

        if (square.getPosition().x > 500)
            square.setPosition(-500, 0);

        window.clear(sf::Color::White);
        window.draw(square);
        window.display();
    }

    return 0;
}
```

Рисунок 4.9 – Переписанный на ЯП C++ код

Список ошибок		
Все решение ✖ 20 Ошибки ⚠ 1 Предупреждение ℹ 0 Сообщения 🔍 Сборка и IntelliSense		
Код	Описание	
⚠ C4305	аргумент: усечение из "double" в "float"	
✖ LNK2001	неразрешенный внешний символ mainSRTStartup.	
✖ LNK2019	ссылка на неразрешенный внешний символ __imp__calloc_dbg в функции "char * __cdecl std::_Maklocstr<char>(char const *,char *,struct _Cvtvec const &)" (??\$_Maklocstr@D@std@@@YAPEADPEBDPEADAEBU_Cvtvec@@@Z).	
✖ LNK2019	ссылка на неразрешенный внешний символ memcpu в функции wmemcpu.	
✖ LNK2019	ссылка на неразрешенный внешний символ __imp_wcslen в функции "wchar_t * __cdecl std::_Maklocwcs(wchar_t const *)" (?_Maklocwcs@std@@@YAPEA_WPEB_W@Z).	
✖ LNK2019	ссылка на неразрешенный внешний символ strlen в функции "char * __cdecl std::_Maklocstr<char>(char const *,char *,struct _Cvtvec const &)" (??\$_Maklocstr@D@std@@@YAPEADPEBDPEADAEBU_Cvtvec@@@Z).	
✖ LNK2001	неразрешенный внешний символ __CxxFrameHandler4.	
✖ LNK2001	неразрешенный внешний символ __CxxFrameHandler4.	
✖ LNK2019	ссылка на неразрешенный внешний символ _CrtDbgReport в функции _CRT_RTC_INIT.	
✖ LNK2019	ссылка на неразрешенный внешний символ _CrtDbgReportW в функции _CRT_RTC_INITW.	
✖ LNK2019	ссылка на неразрешенный внешний символ strcpy_s в функции "void __cdecl _RTC_StackFailure(void *,char const *)" (?_RTC_StackFailure@@@YAXPEAXPEBD@Z).	
✖ LNK2019	ссылка на неразрешенный внешний символ strcat_s в функции "void __cdecl _RTC_StackFailure(void *,char const *)" (?_RTC_StackFailure@@@YAXPEAXPEBD@Z).	
✖ LNK2019	ссылка на неразрешенный внешний символ __stdio_common_vsprintf_s в функции _vsprintf_s_l.	
✖ LNK2001	неразрешенный внешний символ __C_specific_handler_noexcept.	

Рисунок 4.10 – Ошибки при попытке запуска скомпилированного кода на ЯП

C++

4.2. ПРИМЕНЕНИЕ СЕРВИСА AI CODE CONVERTER

В данном разделе рассмотрим возможность конвертации кода с помощью сервиса AI Code Converter. Данный сервис позволяет переводить код не только с ЯП Python, но также и со множества других ЯП. В нашем случае попробуем перевести 3 программы из предыдущего раздела в код на ЯП C++.

Первой переведём программу, проверяющую, является ли слово палиндромом. Код программы и результат выполнения были приведены на Рисунке 4.1. Теперь запросим сервер перевести этот код на ЯП C++ (Рисунок 4.11).

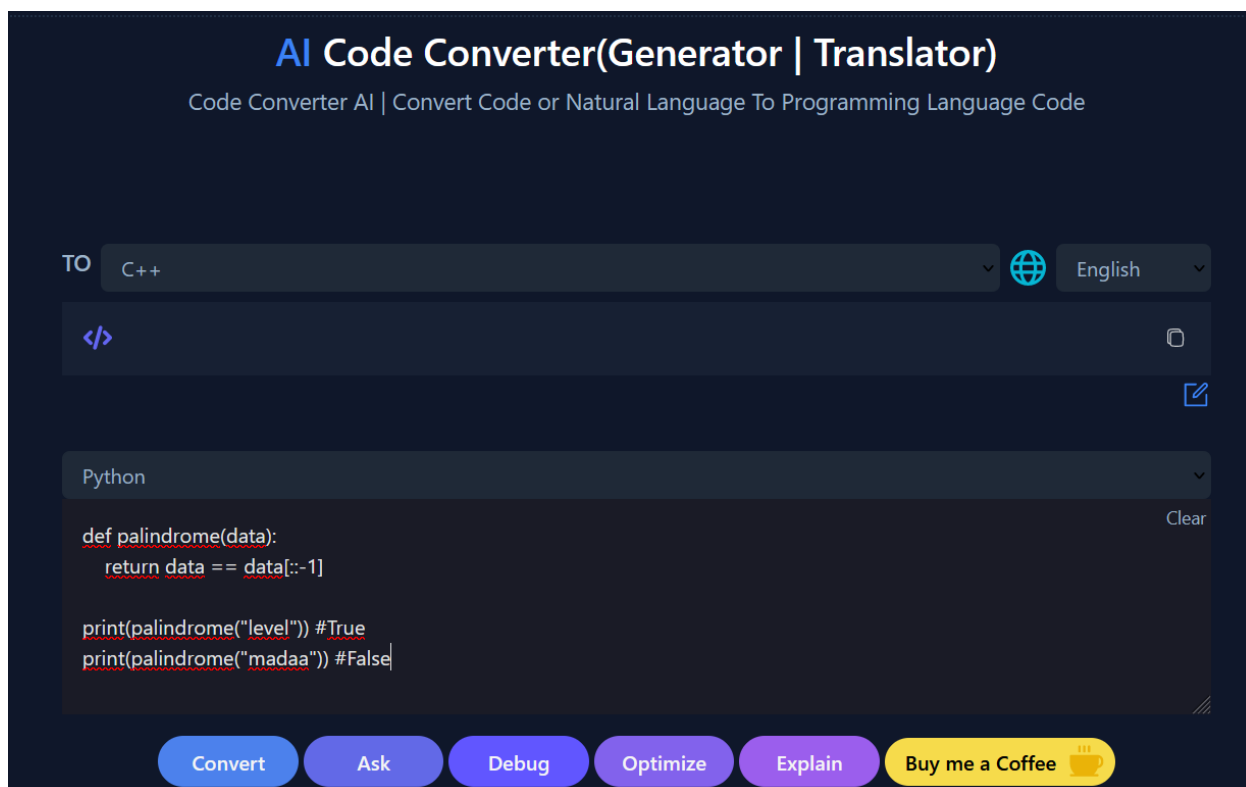
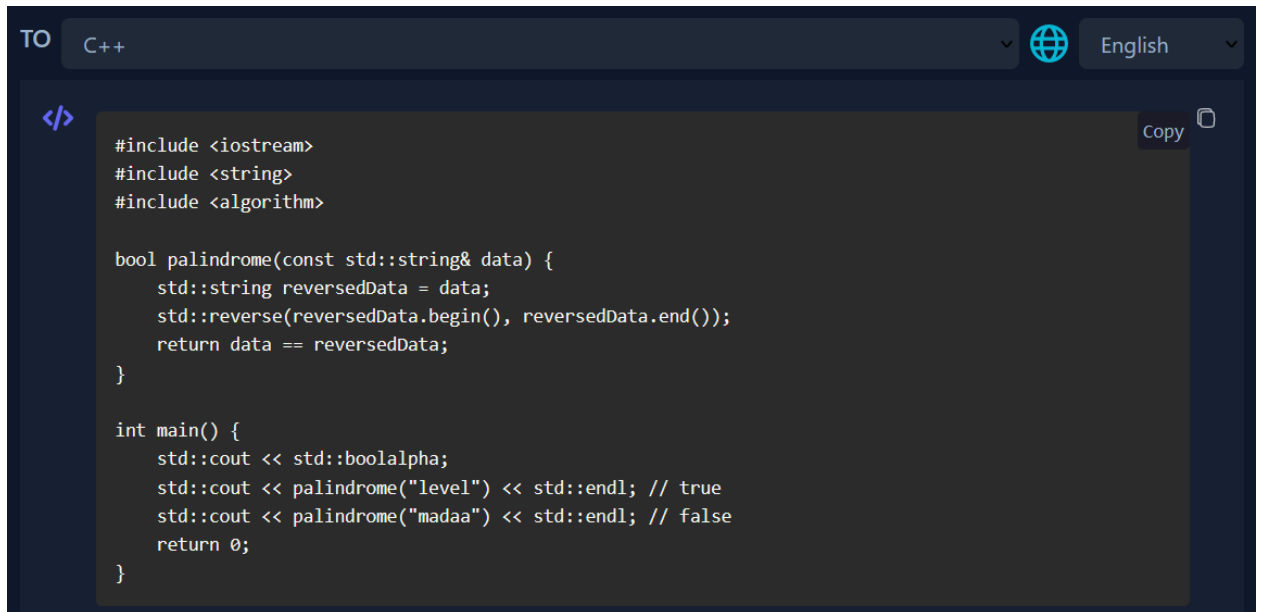


Рисунок 4.11 – Общий вид сервиса

Для конвертации необходимо ввести код в поле внизу, после чего нажать кнопку «Convert». В результате в поле сверху получим готовый код (Рисунок 4.12). Запишем полученный код в IDE и выполним. Как видно из

Рисунка 4.13, программа успешно выполнилась и выдала ожидаемый результат.

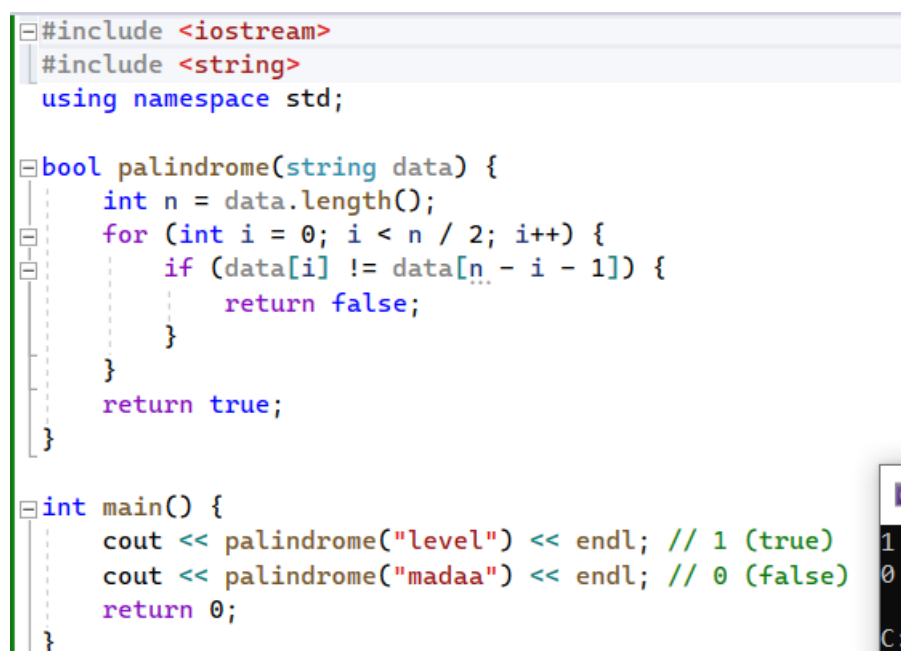


```
TO C++ English
</>
#include <iostream>
#include <string>
#include <algorithm>

bool palindrome(const std::string& data) {
    std::string reversedData = data;
    std::reverse(reversedData.begin(), reversedData.end());
    return data == reversedData;
}

int main() {
    std::cout << std::boolalpha;
    std::cout << palindrome("level") << std::endl; // true
    std::cout << palindrome("madaa") << std::endl; // false
    return 0;
}
```

Рисунок 4.12 – Готовый код в сервисе

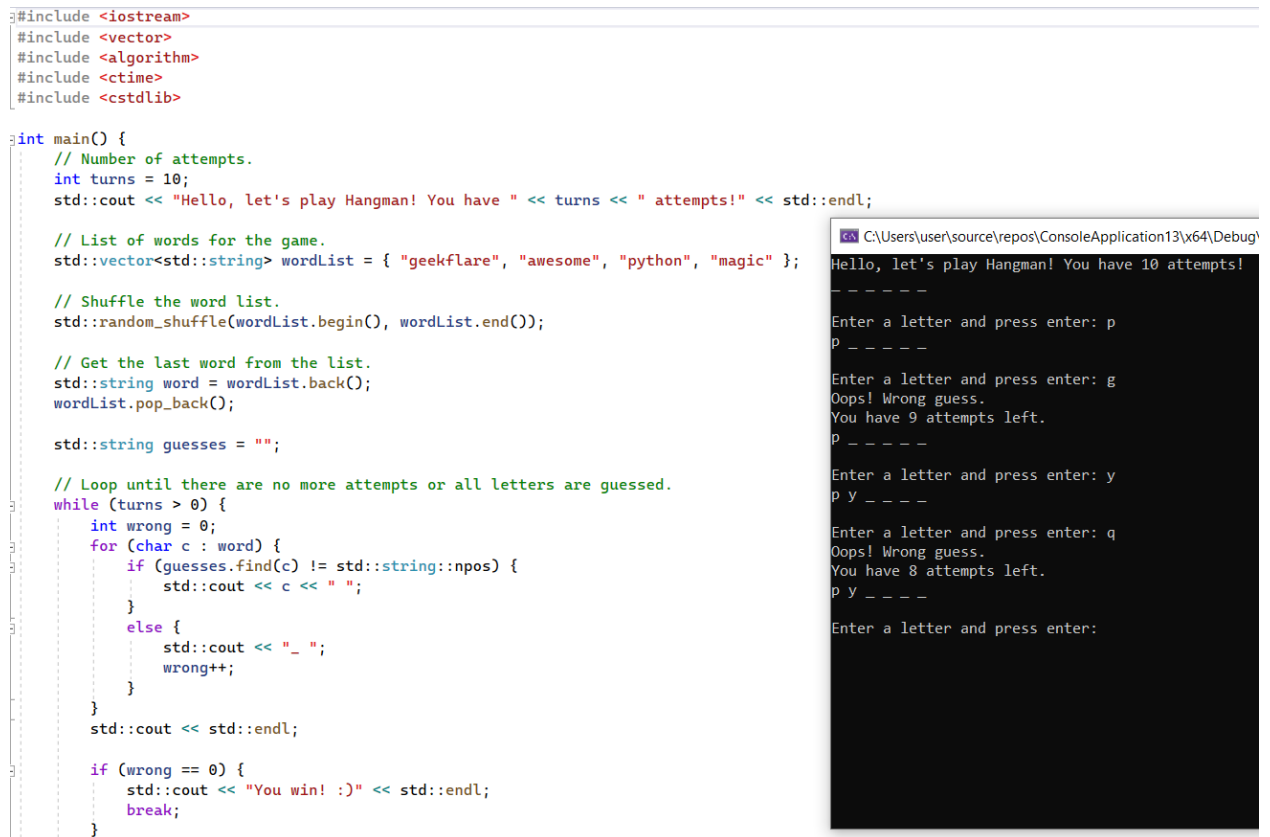


```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 bool palindrome(string data) {
6     int n = data.length();
7     for (int i = 0; i < n / 2; i++) {
8         if (data[i] != data[n - i - 1]) {
9             return false;
10        }
11    }
12    return true;
13 }
14
15 int main() {
16     cout << palindrome("level") << endl; // 1 (true)
17     cout << palindrome("madaa") << endl; // 0 (false)
18     return 0;
19 }
```

Рисунок 4.13 – Выполненный код в IDE

Таким образом, простую программу сервис легко переводит.

Возьмём следующую по сложности программу – консольную программу для игры в «виселицу», код которой представлен на Рисунке 4.4. По аналогии, получим код на выходе сервиса и запустим его в IDE. Результат приведён на Рисунке 4.14.

The image shows a C++ source code file on the left and its execution output in a terminal window on the right. The code is a Hangman game implementation. It includes headers for iostream, vector, algorithm, ctime, and cstdlib. The main function sets up a word list, shuffles it, and starts a loop where the user guesses letters. The terminal output shows the game in progress, with the user guessing 'p', 'g', and 'q', receiving feedback on correctness and remaining attempts. The game title is 'Hello, let's play Hangman! You have 10 attempts!'.

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <ctime>
#include <cstdlib>

int main() {
    // Number of attempts.
    int turns = 10;
    std::cout << "Hello, let's play Hangman! You have " << turns << " attempts!" << std::endl;

    // List of words for the game.
    std::vector<std::string> wordList = { "geekflare", "awesome", "python", "magic" };

    // Shuffle the word list.
    std::random_shuffle(wordList.begin(), wordList.end());

    // Get the last word from the list.
    std::string word = wordList.back();
    wordList.pop_back();

    std::string guesses = "";

    // Loop until there are no more attempts or all letters are guessed.
    while (turns > 0) {
        int wrong = 0;
        for (char c : word) {
            if (guesses.find(c) != std::string::npos) {
                std::cout << c << " ";
            }
            else {
                std::cout << "_ ";
                wrong++;
            }
        }
        std::cout << std::endl;

        if (wrong == 0) {
            std::cout << "You win! :)" << std::endl;
            break;
        }
    }
}
```

C:\Users\user\source\repos\ConsoleApplication13\x64\Debug\
Hello, let's play Hangman! You have 10 attempts!
_ _ _ _ _
Enter a letter and press enter: p
p _ _ _ _
Enter a letter and press enter: g
Oops! Wrong guess.
You have 9 attempts left.
p _ _ _ _
Enter a letter and press enter: y
p y _ _ _
Enter a letter and press enter: q
Oops! Wrong guess.
You have 8 attempts left.
p y _ _ _
Enter a letter and press enter:

Рисунок 4.14 – Запуск конвертированной программы

Как можно заметить, программа скомпилировалась и весь функционал, заложенный в неё, работает. Следует отметить, что в отличии от предыдущего способа, представленного в разделе 4.1, программа автоматически перевела текст программы на английский язык, тем самым, избавив нас от исправления ошибки, с которой мы сталкивались ранее. Однако, если посмотреть внимательнее, то конвертация на другой язык происходит по желанию пользователя, и по умолчанию выбран английский. Давайте исправим это. Поле с данной настройкой выделено на Рисунке 4.15.

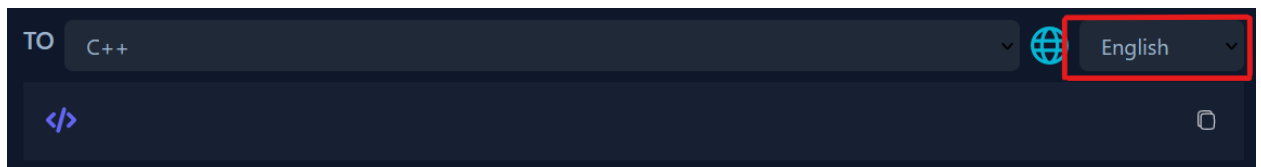


Рисунок 4.15 – Поле выбора языка

Снова выполним конвертацию кода с обновлёнными настройками. Результат, также как и предыдущий, скомпилируем и запустим. Полученный вывод изображён на Рисунке 4.16.

```
Привет, Давай сыграем в виселицу! У тебя есть 10 попыток!
-----
Впиши букву и нажми enter: _
```

Рисунок 4.16 – Вывод программы со строками на русском языке

Как видно, мы столкнулись с той же проблемой. Сервис не добавил локализацию на русский язык. Исправим это, добавив необходимую строку кода «setlocale(LC_ALL, “ru”);». После того, как добавим данную строку, попробуем снова запустить программу, что отобразим на Рисунке 4.17.

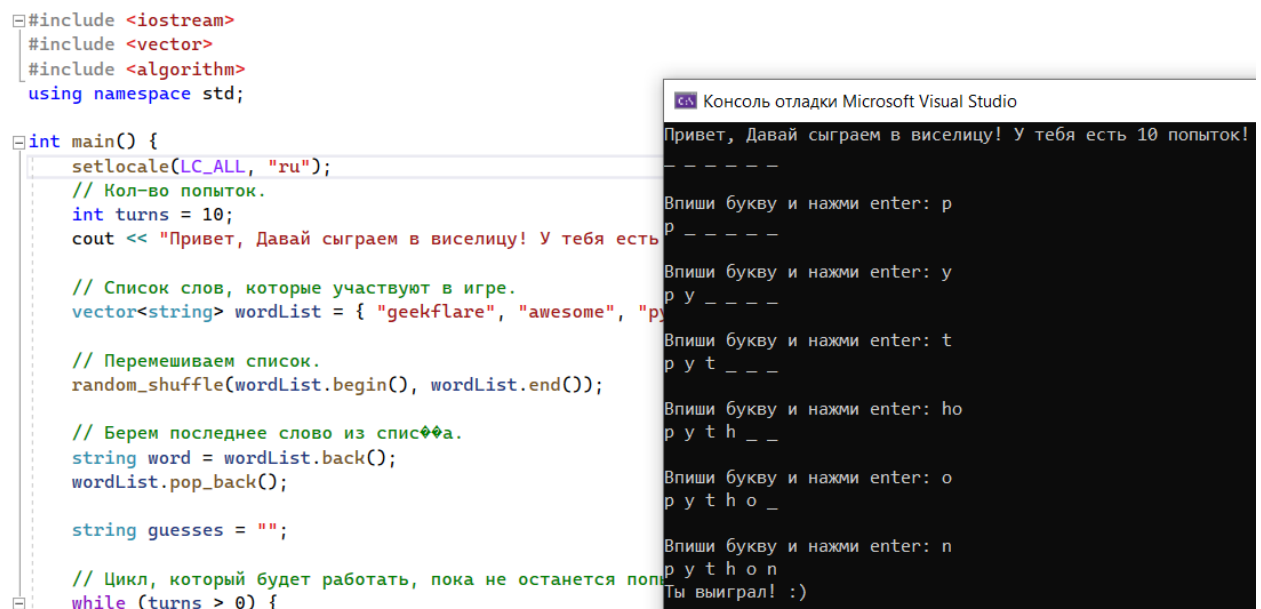
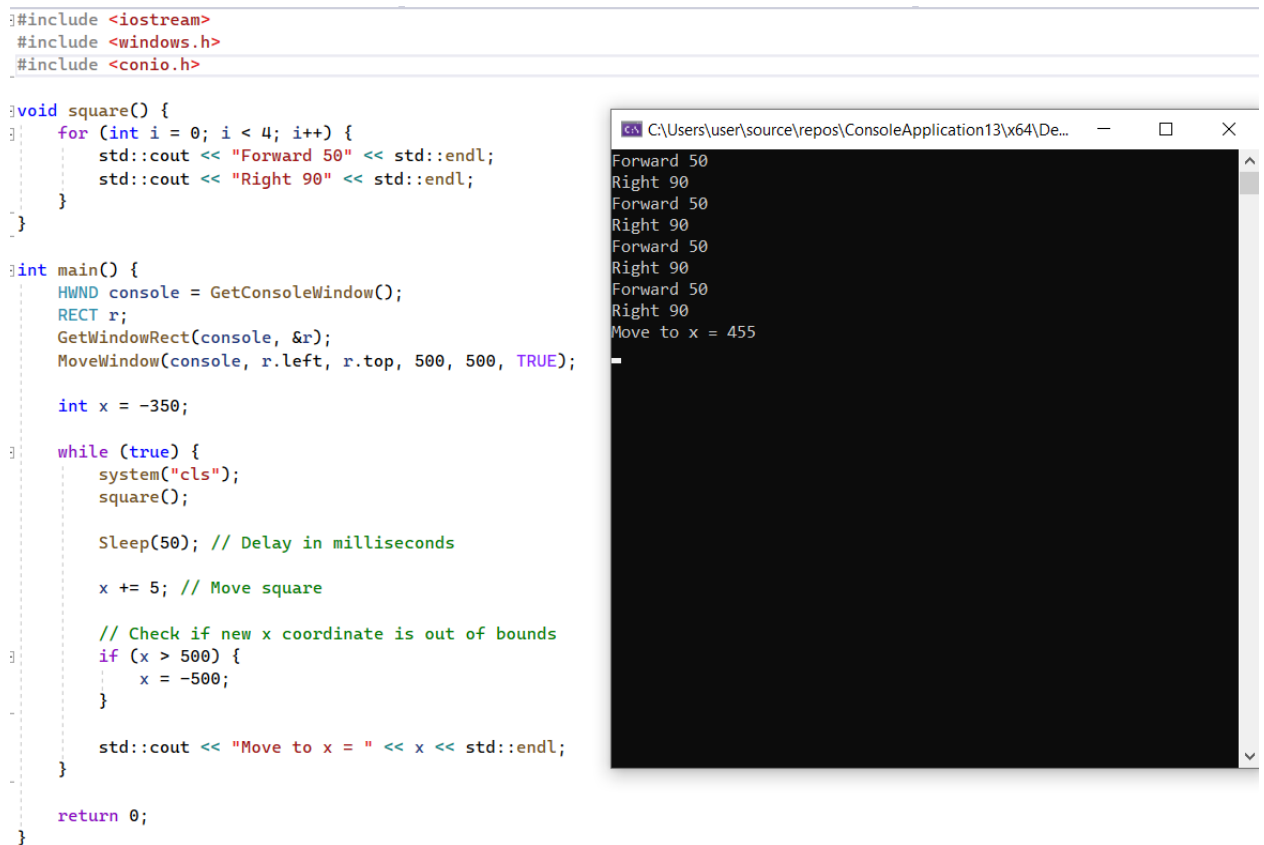


Рисунок 4.17 – Запуск сконвертированной программы с русской локализацией

Наша программа работает так, как и ожидается.

Перейдём к самому сложному – конвертации программы «трудного» уровня, использующей графику и стороннюю библиотеку. Этот пример возьмём также, как и предыдущие, из раздела 4.1. Код приведён ранее на Рисунке 4.8. Результат конвертации приведён на Рисунке 4.18. Как можно видеть, конвертация оказалась провальной. Несмотря на попытку использовать работу с графикой, в данном случае сервис лишь изменил размер окна консоли. При этом программа работает без нареканий, с оговоркой на то, что утерян весь исходный функционал. На смену ему появился бесконечный цикл, обновляющий переменную x.



```
#include <iostream>
#include <windows.h>
#include <conio.h>

void square() {
    for (int i = 0; i < 4; i++) {
        std::cout << "Forward 50" << std::endl;
        std::cout << "Right 90" << std::endl;
    }
}

int main() {
    HWND console = GetConsoleWindow();
    RECT r;
    GetWindowRect(console, &r);
    MoveWindow(console, r.left, r.top, 500, 500, TRUE);

    int x = -350;

    while (true) {
        system("cls");
        square();

        Sleep(50); // Delay in milliseconds

        x += 5; // Move square

        // Check if new x coordinate is out of bounds
        if (x > 500) {
            x = -500;
        }

        std::cout << "Move to x = " << x << std::endl;
    }

    return 0;
}
```

C:\Users\user\source\repos\ConsoleApplication13\x64\De... Forward 50
Right 90
Forward 50
Right 90
Forward 50
Right 90
Forward 50
Right 90
Forward 50
Right 90
Move to x = 455

Рисунок 4.18 – Результат выполнения сконвертированного «сложного» кода

В рамках данного раздела рассмотрен способ конвертации кода с ЯП Python на ЯП C++. Было рассмотрено три примера разной сложности. По

результатам можно утверждать, что конвертация кода с помощью стороннего инструментария способна качественно помочь перенести простой по логике код на другой ЯП с минимальными доработками или вовсе без них. С другой стороны, конвертация кода со сложной логикой до сих пор является трудной задачей, с решением которой не способен справиться инструментарий, рассматриваемый в данном разделе, и для качественного перевода требуется квалифицированный человек [12].

Таблица 4.1 – Сравнительная характеристика способов конвертации

Критерии сравнения	Способ конвертации		
	GPT-3.5	AI Code Converter	Ручная конвертация
Скорость перевода	Высокая	Средняя	Низкая
Возможность конвертации стандартных библиотек	Да	Да	Да
Возможность конвертации сторонних библиотек	Частично	Нет	Да
Возможность локализации программы	Да	Частично	Да
Поддержка оптимизации кода	Частично	Частично	Да
Себестоимость перевода	В общем случае, ниже, чем при ручной	В общем случае, ниже, чем при ручной	Высокая
Необходимость анализа и правок полученного кода	Да	Да	Частично

ЗАКЛЮЧЕНИЕ

В данном учебном пособии рассмотрены несколько способов проведения реверс-инжиниринга программ, как созданных для конкретных ЯП, так и универсальных. Затронута тема конвертации программного кода с одного ЯП на другой. На примерах показано, что данное направление в текущее время недостаточно развито и имеет перспективы в будущем, в частности, за счёт возможностей нейронных сетей.

Базовые знания, изложенные в пособии, помогут снизить порог вхождения в работу с инструментарием обратной разработки. Рассмотрены методы поиска ключевых мест в программах. Отдельно освещены дополнительные темы: источники практических заданий, документация к программам, перспективы в изучении данных инструментов.

Методы, представленные в данном учебном пособии, хоть и распространены, но не являются универсальными. Полное понимание процессов, происходящих в программе – зачастую затратное по времени, или в целом невозможное для человека действие. Однако хорошее понимание механик, основанных на изложенных в данном пособии материалах, способствует ускорению работы и отладки программ.

Происходит постоянная «гонка вооружений», заключающаяся в создании новых способов защиты и их обхода. Для того, чтобы выполнить комплекс работ и в будущем стать квалифицированным специалистом, необходимо постоянное совершенствование в данной области.

СПИСОК ЛИТЕРАТУРЫ

1. Савин, Л. А. Применение инструментов обратной разработки для выявления недеklarированных возможностей программного обеспечения систем управления / Л. А. Савин // Проблемы управления безопасностью сложных систем : Материалы XXXI международной конференции, Москва, 13 декабря 2023 года. – Москва: Институт проблем управления им. В.А. Трапезникова РАН, 2023. – С. 412-419. – DOI 10.25728/iccscs.2023.49.69.057. – EDN BDVJGP.
2. Веб-сервис для хостинга IT-проектов GitHub [Электронный ресурс] // GitHub : The complete developer platform to build, scale, and deliver secure software. – 2023. –URL: <https://github.com/NotPrab/.NET-Deobfuscator> (дата обращения 24.10.23).
3. Инструменты реверс-инжиниринга: Учебное пособие/ Домашкин А.Д., Костенко С.А., Николаев А.М. и др. – РУТ(МИИТ).2023. – 67 с.
4. icsharpcode/ILSpy: .NET Decompiler with support for PDB generation, ReadyToRun, Metadata (&more) // GitHub URL: <https://github.com/icsharpcode/ILSpy> (дата обращения: 28.01.24).
5. Crackmes // crackmes.one URL: <https://crackmes.one/crackme/64d5ef92b25df8732eebc74c> (дата обращения: 18.11.23).
6. NET Decompiler: Decompile Any .NET Code | .NET Reflector // RedGate URL: 1. <https://www.red-gate.com/products/reflector/> (дата обращения: 18.11.23).
7. NationalSecurityAgency/ghidra // GitHub URL: <https://github.com/NationalSecurityAgency/ghidra/releases> (дата обращения: 28.01.24).
8. OpenJDK JDK 21.0.2 GA Release // jdk.java.net URL: <https://jdk.java.net/21/> (дата обращения: 28.01.24).

9. Crackmes // crackmes.one URL:
<https://crackmes.one/crackme/65ad74aceef082e477ff5f24> (дата обращения:
28.01.24).

10. Dark Theme ASCII Table // ByteTool URL:
<https://bytetool.web.app/en/ascii/> (дата обращения: 28.01.24).

11. Crackmes // crackmes.one URL:
<https://crackmes.one/crackme/65ba60e0eef082e477ff6482> (дата обращения:
29.02.24).

12. Савин, Л. А. Конвертация искусственных языков / Л. А. Савин // Цифровые технологии и решения в сфере транспорта и образования : материалы II Национальной научно-практической конференции, Москва, 07 декабря 2023 года. – Москва: Российский университет транспорта, Белый ветер, 2023. – С. 172-177. – EDN OHFJNY.

Ковров Артём Игоревич
Ляпина Елизавета Павловна
Савин Лев Андреевич
Сидоренко Валентина Геннадьевна
Соколов Илья Дмитриевич

Инструменты реверс-инжиниринга и транскомпиляции

Учебное пособие